

HTML Notes

Class notes for the 1st Semester

Are you ready to learn HTML and web technologies? Press `space` on your keyboard →





Table of contents

1. [Software Engineering](#)
2. [Why Web and Cloud?](#)
3. [Focus of this Class](#)
4. [Tools](#)
5. [Accounts](#)
6. [Installations](#)
7. [Algorithms & Flowcharts](#)
8. [Programming Paradigms](#)
9. [How the Web Works](#)
10. [Getting Started](#)
11. [Document Structure](#)
12. [Semantic HTML](#)
13. [Heading & Sections](#)
14. [Attributes](#)
15. [Text Basics](#)
16. [Links](#)
17. [Navigation](#)
18. [Tables](#)
19. [Forms](#)
20. [HTML API](#)
21. [Focusing](#)
22. [Details & Summary](#)
23. [Dialogs & Popovers](#)
24. [Web Components](#)
25. [Do you know?](#)
26. [Assignments](#)
27. [Terminal](#)
28. [Git](#)
29. [GitHub](#)
30. [Open Source](#)

 [All Sections](#)

 [HTML Playground](#)

 [JS/TS Playground](#)

 [Changelog](#)



What is Software Engineering?

Software Engineering is the systematic application of engineering principles to the design, development, maintenance, testing, and evaluation of software.

It involves a disciplined approach to analyzing user needs, planning and managing projects, creating software systems, and ensuring their reliability, efficiency, and maintainability.

You will need a blend of technical Skills, engineering methods and project management to produce high-quality software systems.

Read more about Software Engineering?

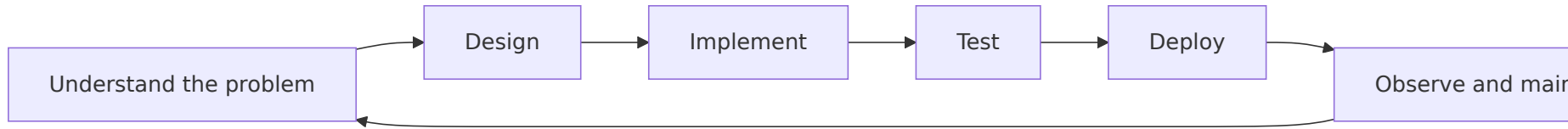


Foundational Requirement

- coding

Software Engineering Is More Than Coding

Coding is one part of building software that remains useful, reliable, secure, and understandable over time.



- **Requirements:** What problem are we solving, and for whom?
- **Quality:** Does it work correctly, safely, and accessibly?
- **Collaboration:** Can another engineer understand, review, and improve it?



A Practical Engineering Workflow

1. Clarify the user need and define what “done” means.
2. Break the problem into small, testable tasks.
3. Design the simplest suitable solution.
4. Implement and verify one small change at a time.
5. Use Git, code review, tests, and documentation to protect quality.
6. Deploy, observe real usage, fix problems, and improve.

Info

AI can assist at every step, but the engineer remains responsible for the requirements, decisions, verification, and outcome.



Why Web and Cloud?

- Career Advantages
 - High Demand and Competitive Salaries
 - Remote Work
 - Entrepreneurial Options
- Technical Advantages
 - Scalability, Cost Efficiency, Security and CI/CD
 - Access to Advanced Tools and Services
 - Global Reach
- Educational and Community Benefits
 - Extensive Learning Resources
 - Active Community Support and Open Source
- Innovation and Future-Proofing
 - Cutting-Edge Technologies
 - Adaptability



Focus of this Class

-  HTML
-  CSS
-  JavaScript and  TypeScript
-  Git and  GitHub | Open Source | Introduction to basic terminal commands



Tools

-  Visual Studio Code or any equivalent (JetBrains  Webstorm IDE,  Zed, Sublime Text), Cursor
- Git and GitBash for windows,  Git only for mac and linux
-  Nodejs,  Python

Here is a link to watch a video on installation and basic set of vs code, git on windows operating system.



Accounts

Some Account You Expected to (Create || Have)

-  GitHub and any other equivalent in GitLab or BitBucket
- LinkedIn
- Twitter or X
-  Stackblitz or  Codesandbox
-  Codepen
- Stackoverflow
-  ChatGPT
-  Figma
- Dev.to | hashnode | Medium
-  Slack or  Discord
-  Netlify |  Vercel |  Render
- Personal Website. we will teach to create your own.



Installations



Set up your VSCODE for window users-altschool

Oluwasetemi Ojo

SET UP YOUR
VS CODE
ON WINDOWS OS



ALTSCHOOL CLASS VERSION



Watch on  YouTube

Algorithms and FlowCharts

Definition: An algorithm is a step-by-step procedure or a set of rules designed to perform a specific task or solve a particular problem. It is a sequence of instructions that are followed to achieve a desired outcome.

Example: An algorithm for making a cup of tea might include steps like boiling water, adding a tea bag to a cup, pouring the hot water into the cup, letting it steep for a few minutes, and then removing the tea bag.



Tip

1. Patterns Reduce Complexity like two-pointer, sliding window, divide and conquer, dynamic programming, etc.
2. Optimize Thinking for Efficiency
3. Develop Transferable Skills
4. Practice and Apply



Success

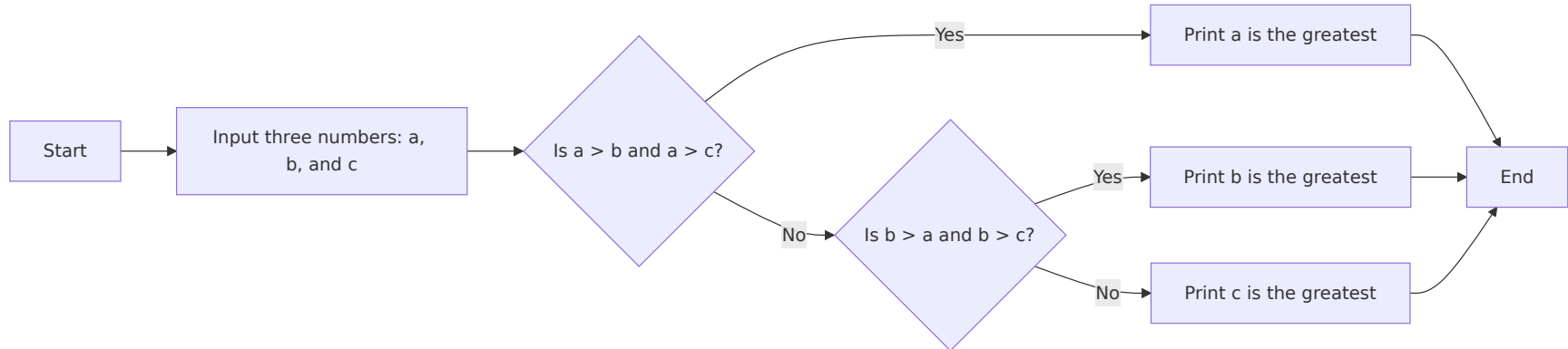
Building intuition around patterns will significantly improve your problem-solving skills, speed, and confidence in tackling algorithmic challenges.

flowchart

Definition: A flowchart is a visual representation of the steps in a process or system using symbols, arrows, and text. It depicts the sequence of operations or steps, making it easier to understand how a process flows from start to finish.

Example: A flowchart for logging into a website might start with a "Start" symbol, followed by a decision symbol asking if the user has entered their username and password, arrows leading to "Enter username" and "Enter password" steps, and an end symbol once the login process is successful.

Flowchart for finding the greatest of three numbers a, b, and c





Problem Solving

Use  ChatGPT or Google to research what the key components of problem solving are. Try to use it to solve any problem you have.



Programming Paradigms

- Procedural Programming
- Object-Oriented Programming (OOP)
- Functional Programming
- Logic Programming
- Declarative Programming
- Concurrent Programming
- Event-Driven Programming

Each paradigm brings its own way of thinking and problem-solving, making some paradigms more suitable for certain types of tasks than others. Modern programming often involves a combination of these paradigms to leverage their respective strengths.



Programming Concepts

- Variables
- Data Types
- Control Structures
- Functions (or Methods)
- Data Structures
- Algorithms
- Object-Oriented Concepts
 - Encapsulation
 - Polymorphism
 - Inheritance
- Recursion
- Error Handling
- Memory Management
- Concurrency
- File I/O

Understanding these concepts is fundamental to mastering programming and can significantly improve your ability to develop complex and efficient software solutions.



Helpful resources

Use these ByteByteGo visual guides to review concepts from these notes:

- **Software development lifecycle / workflow:** [How do Companies Ship Code to Production?, CI/CD Pipeline Explained in Simple Terms, DevOps vs NoOps](#)
- **Code quality and engineering habits:** [10 Good Coding Principles to Improve Code Quality](#)
- **Programming paradigms:** [Top 8 Programming Paradigms, Imperative vs Functional vs Object-oriented Programming](#)
- **Object-oriented programming:** [The Fundamental Pillars of Object-Oriented Programming](#)
- **Data structures and algorithms:** [10 Key Data Structures We Use Every Day, The 9 Algorithms That Dominate Our World](#)
- **Concurrency and memory management:** [Concurrency vs Parallelism, How does Garbage Collection work?](#)
- **Web, DNS, URL, and HTTP:** [How Does DNS Lookup Work?, URL, URI, URN - Differences Explained, HTTP Status Codes You Should Know, Top 9 HTTP Request Methods](#)
- **Browser and web application basics:** [How Browsers Render Web Pages, 10 Essential Components of a Production Web Application](#)



Variables

Storage locations in memory with a name, used to hold data.

```
1 let name = 'AltSchool'  
2 let age = 99  
3  
4 console.log({name, age})
```

```
{  
  "name": "AltSchool",  
  "age": 99  
}
```



```
1 name = 'AltSchool'  
2 age = 99  
3  
4 print(name, age)
```

Data Types

Classification of data items, defining the operations that can be performed on them.

Primitive types: int, char, float, boolean, number, bigint, symbol, string, undefined, null,

Composite types: arrays, structs, classes.

Abstract data types: List, Stack, Queue, etc.

```
1 let name = 'AltSchool'
2 let age = 99
3 let isStudent = true
4
5 const arrayOfScore = [99, 40, 50]
6 const person = { name: name, age: age, isStudent: isStudent }
7
8 console.log(arrayOfScore)
9 console.log(person)
```

```
[99, 40, 50]
{
  "name": "AltSchool",
  "age": 99,
  "isStudent": true
}
```





Control Structures

Direct the order of execution of statements in a program.

Conditional statements: if, else, switch.

Loops: for, while, do-while.

Branching: break, continue, return

```
1 for (let index = 0; index < array.length; index++) {  
2   const element = array[index];  
3 }  
4  
5 while (condition) {  
6   // code block to be executed  
7 }
```



Functions (or Methods)

Blocks of code designed to perform a particular task, reusable throughout the program.

Key Points

- Definition and calling.
- Parameters and return values.
- Scope and lifetime of variables.



Data Structures

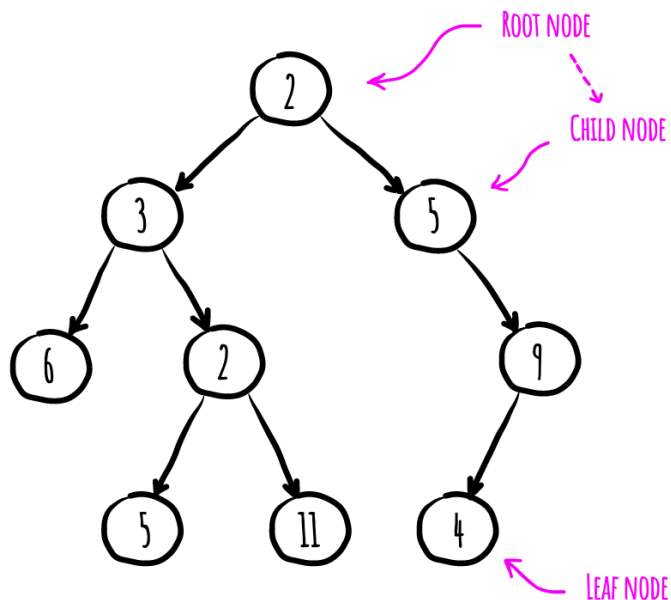
Ways of organizing and storing data to enable efficient access and modification

Key Points: Linear: Arrays, Linked Lists. Non-linear: Trees, Graphs. Abstract: Stack, Queue, Map, Set

Data Structures More (Trees)

Binary Search Tree, AVL Tree, Red-Black Tree Segment Tree Fenwick Tree (Binary Indexed Tree)

TREE



A COLLECTION OF NODES, WHERE EACH NODE IS A DATA STRUCTURE CONSISTING OF A VALUE, TOGETHER WITH A LIST OF REFERENCES TO NODES, WITH THE CONSTRAINTS THAT:

- NO REFERENCE IS DUPLICATED,
- AND NONE POINTS TO THE ROOT.

THERE ARE MANY KINDS OF TREES:





Algorithms

Step-by-step procedures or formulas for solving problems.

Key Points

Sorting: Bubble sort, Quick sort, Merge sort. Searching: Linear search, Binary search. Complexity: Big O notation for time and space.



Object-Oriented Concepts

Principles used in OOP to create objects that model real-world entities

Key Points

Classes and Objects. Encapsulation, Inheritance, Polymorphism, Abstraction. Constructors and destructors

Encapsulation - Hiding the internal state and requiring all interaction to be performed through an object's methods

Polymorphism - The ability of different classes to be treated as instances of the same class through a common interface

Inheritance - is a mechanism that allows a class to inherit properties and behaviors from another class



Recursion

A function calling itself to solve a smaller instance of the same problem.

Notable Key Points

- Base case and recursive case.
- Stack overflow and efficiency considerations.
- Examples: Factorial, Fibonacci sequence



Error Handling

Mechanisms to handle runtime errors or exceptional conditions.

Key Points

- Try, catch, finally blocks.
- Throwing exceptions.
- Custom exception classes



Memory Management

Techniques to control the allocation, use, and deallocation of memory

Concurrency and Asynchronous Operations

Running multiple computations simultaneously

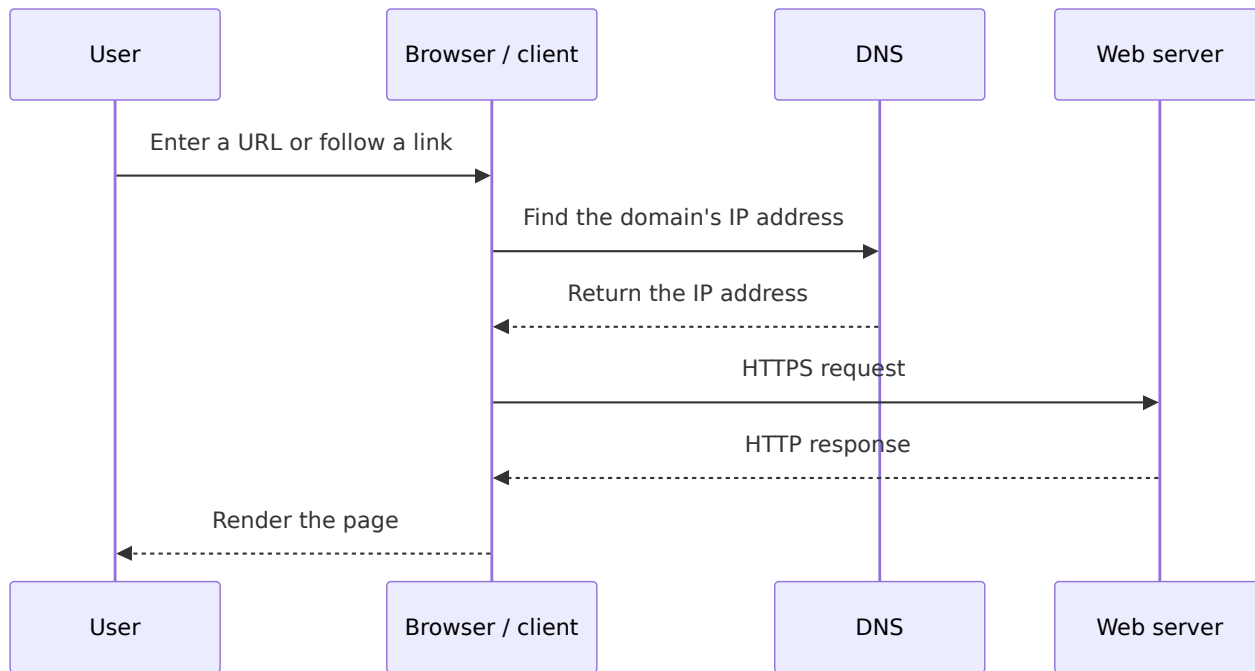
File I/O

Reading from and writing to files. File streams, Opening, reading, writing, and closing files, Binary vs text files



How the Web Works

The internet is the global network connecting computers. The Web is a service that uses that network to exchange linked documents and application data.





From URL to Page

- A URL identifies a resource: `https://example.com/about?lang=en#team`.
- DNS translates the domain name into an IP address.
- The browser opens a secure HTTPS connection to the server.
- It sends an HTTP request such as `GET /about`.
- The server handles the request and sends a response.
- The browser requests linked assets, builds the page, and renders it.



HTTP Requests and Responses

An HTTP message has a starting line, headers containing metadata, and sometimes a body containing data.

```
1 GET /about HTTP/1.1
2 Host: example.com
3
4 HTTP/1.1 200 OK
5 Content-Type: text/html
6
7 <!doctype html> ...
```

- A request states what the client wants to do and which resource it needs.
- A response contains a status code, headers, and usually HTML, JSON, an image, or another resource.
- Common status codes: **200** success, **301** redirect, **404** not found, and **500** server error.



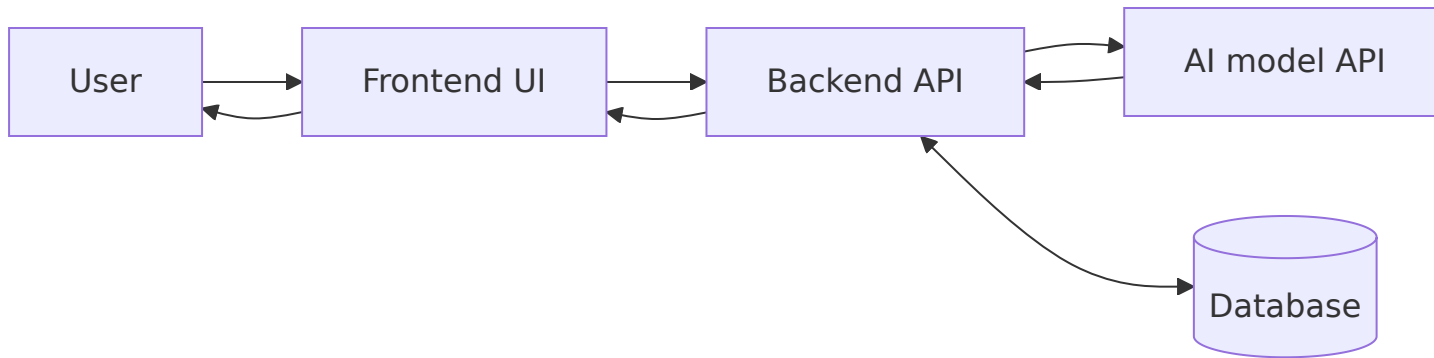
The Parts of a Web Application

Part	Responsibility
HTML	Content and meaning
CSS	Presentation and layout
JavaScript	Behaviour and interaction
Frontend	The application running in the browser
Backend	Business logic, authentication, and server-side APIs
Database	Persistent application data
Deployment	Making the application available to users

A **static** site can return prepared files. A **dynamic** application generates or retrieves data for each request, often through an API using JSON.



Where AI Fits in a Full-stack Application



- The frontend collects input and presents results.
- The backend validates requests, applies business rules, and calls the model securely.
- The database stores only the data the application is permitted to retain.
- Model output is untrusted input: validate it before displaying, storing, or acting on it.

Danger

Never place AI provider secrets or private API keys in browser HTML or client-side JavaScript.



Responsible Use of AI

- Treat generated answers and code as suggestions—not verified facts.
- Test the result and understand it before submitting or deploying it.
- Do not share passwords, API keys, personal data, or proprietary code with an AI tool.
- Review generated interfaces for accessibility, security, and maintainability.
- Record important assumptions and decisions so teammates can review them.
- Follow programme, employer, and client rules for attribution and acceptable AI use.

Info

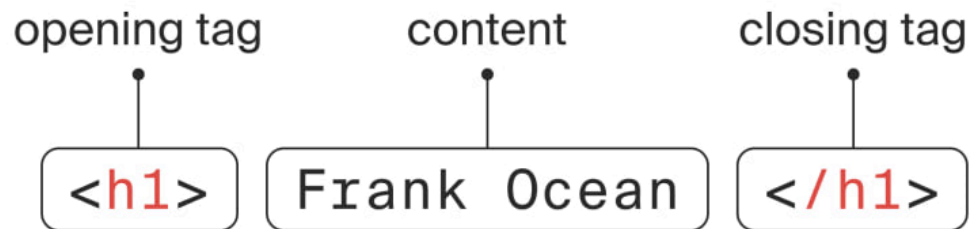
AI can increase your speed; engineering judgment determines whether the result is correct and safe.



HTML(HyperText Markup Language) is the foundation of basically every web page, basically, it is the core language of the World Wide Web. It's how we tell browsers to structure content into paragraphs, headings, images, links, lists, forms, tables, buttons, and more. If you're interested in building a website, web development, or just coding in general, learning HTML is a great place to start.

Getting Up and Running with HTML

Getting Up and Running with HTML, its important to understand the basic structure of an HTML document. The `Document Type` and HTML content itself (which is the `html` element). This contains the Metadata (`head` element and its related elements) and Body, which is what is displayed in the browser's visible area.





Let's break it down a bit

- HTML uses "markup" to annotate text, images, and other content for display in a Web browser. This markup is called ELEMENT or TAG. HTML Element or Tag includes `<head>`, `<title>`, `<body>`, `<header>`, `<footer>`, `<article>`, `<section>`, `<p>`, `<div>`, `<span>`, `<img>`, `<aside>`, `<audio>`, `<canvas>`, `<datalist>`, `<details>`, `<embed>`, `<nav>`, `<search>`, `<output>`, `<progress>`, `<video>`, `<ul>`, `<ol>`, `<li>` and many others.
- "HyperText" is text on a web page that contains references to another web page. You probably know these as hyperlinks. We use hyperlinks to jump to another section of the same page, a different page on the current website, or a completely new website. Hyperlinks can also open a PDF, email, or multimedia, like a video or audio file.
- Linking information together in this way was a revolutionary step in building the web. Together, HTML and the internet make it possible for anyone to access all types of information around the world, in any order they want.
- Finally, "Language" is the simplest part of the acronym to understand. Like any language, HTML has a unique syntax and alphabet. But what kind of language is it, exactly? It's a markup language.



HTML Element

An HTML element or "tags", which consist of the element name surrounded by `<` and `>` like `<p>`. The name of an element inside a tag is case-insensitive. That is, it can be written in uppercase, lowercase, or a mixture. For example, the `<title></title>` tag can be written as `<Title>`, `</Title>`, `<TITLE>`, `</TITLE>`, or in any other way. However, for convention and recommendation practice is to write tags in lowercase. These tags or elements can be either self closing `<img />` or normal/common once with opening tag `<p>` and a closing tag `</p>`.

Danger

There are three rendering modes in modern browsers: Quirks Mode(Old, pre-standardized HTML CSS behavior), Standards Mode(modern HTML/CSS standards), and Almost Standards Mode (slight compatibility)

Danger

A page will enter into quirks mode if the doctype is not declared or is declared incorrectly. Browser can auto complete tags, but it is recommended to close tags properly. this can lead to unexpected results.

Danger

If you decide to use the opening tag of an element, you have a duty to close the tag if it is not a self-closing tag.



First HTML Document

create a folder and add an `index.html` file either using your vs code or terminal or file explorer. Use the liveserver extension to view the file in the browser.

Info

Always open your vscode on the right folder your created for your project not on the root folder of your computer or desktop or documents or downloads.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, in
6   <title>Responsive Web Page</title>
7   <meta name="description" content="This is a responsiv
8 </head>
9 <body>
10   <h1>Welcome to the world of HTML</h1>
11 </body>
12 </html>
```

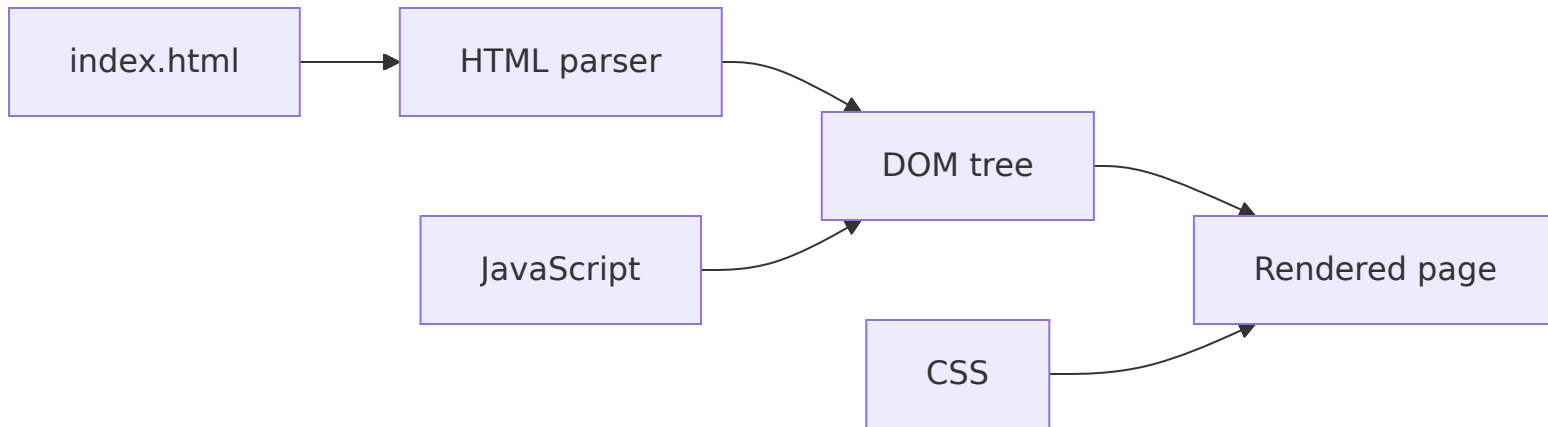


**Welcome to the
world of HTML**



From HTML to the DOM

The browser parses HTML into the **Document Object Model (DOM)**: a tree of objects representing the document. JavaScript and browser tools interact with this tree—not directly with the source file.



Malformed HTML may still render because browsers recover from errors, but the resulting DOM might differ from what you intended.



Browser Developer Tools

Open DevTools with Inspect or your browser's keyboard shortcut `ctrl + shift + i` or `cmd + option + i` (Mac).

- **Elements:** inspect the DOM, attributes, and accessibility information.
- **Network:** inspect requests, responses, status codes, and missing assets.
- **Console:** view browser errors and test small JavaScript expressions.
- **Device mode:** check the page at different viewport sizes.

Suggested debugging loop: reproduce the problem → inspect the DOM and Network panel → read the error → make one change → verify again. Logging, debugger statements, and breakpoints can help you understand the flow of execution.

Warning

Changes made inside DevTools are temporary. Update the source file to keep them.

Paths, Links, and Form Destinations

```
1 project/  
2   |— index.html  
3   |— pages/about.html  
4   |— images/profile.jpg
```

- Relative path from `index.html`: `images/profile.jpg`
- Relative path from `pages/about.html`: `../images/profile.jpg`
- Root-relative path: `/images/profile.jpg`
- Absolute URL: `https://example.com/images/profile.jpg`
- Fragment: `#contact`

The same path rules apply to links, images, stylesheets, scripts, and form `action` URLs. File paths may be case-sensitive after deployment.

Metadata and Discoverability

- Give every page a concise, unique `<title>`.
- Add a useful description for search results and link previews.
- Declare the document language and character encoding.
- Use meaningful headings, landmarks, link text, and image alternatives.
- Ensure important content is present in HTML and is understandable without visual styling.

```
1 <html lang="en">
2 <head>
3   <meta charset="UTF-8">
4   <title>Apply | AI Full-stack Programme</title>
5   <meta name="description" content="Apply for the AI Full-stack career programme.">
6 </head>
```



Validate and Test HTML

- Validate HTML to find invalid nesting, duplicate IDs, and missing required attributes.
- Test keyboard navigation, zoom, narrow screens, links, forms, images, and error states.
- Check the DOM and accessibility tree for the structure the browser created.
- Use the Network panel to find missing resources and unsuccessful requests.
- Fix the first meaningful error, then test again; one mistake can cause several later errors.



Info

Validation checks markup rules. Testing checks whether real users can complete the intended task. A quality page needs both.

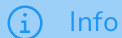


Progressive Enhancement

Start with semantic HTML that delivers the core content and actions. Add CSS for presentation and JavaScript for richer interaction.



- Use links for navigation and buttons for actions.
- Prefer native controls before recreating them with scripts.
- Keep essential tasks usable when a resource fails or is delayed.
- Treat accessibility and resilience as part of the design, not a final patch.

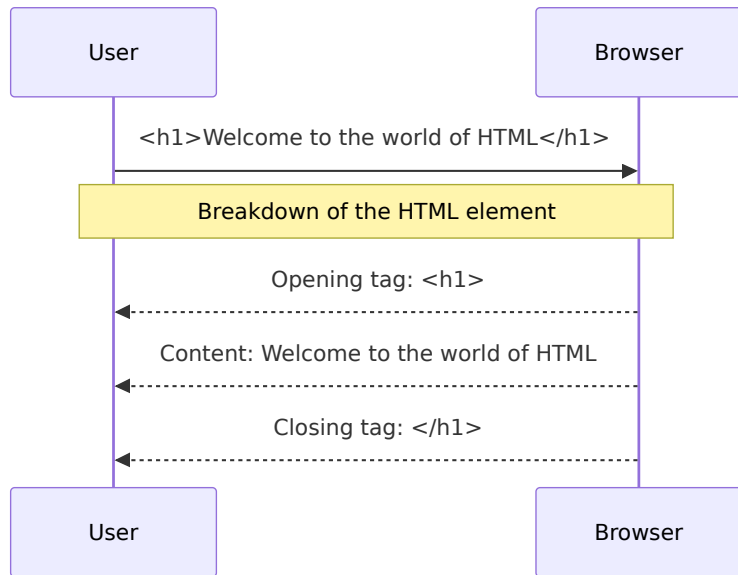


Info

Progressive enhancement keeps the core content and actions usable when styles fail, JavaScript is delayed, or a user relies on assistive technology.



Check this out:



Explanation

The diagram explains what happens when a user view the first html page we created in a browser using right `http://localhost:port` / `http://127.0.0.1:port` if you are using live server vscode extension.

The browser reads the content of the page and displays it in the browser window. The content of the page is enclosed within the `<h1>` tags, which define a top-level heading.

The `h1` is an instruction to the browser to display the text as a top-level heading.

The browser renders or paint the content between the tags on the screen.

The text "Welcome to the world of HTML" is displayed as the main heading of the page.



`<img />`

Self-closing tag



Self Closing Tags like HTML `<img />` Tag

We have been talking about tags in our previous teachings but this time you'll be seeing another form of tag that's called self closing tags under which the popular image tag fall under.

Self-closing tags, also known as void elements, are a feature in HTML and XML where the tag does not require a separate closing tag. Instead, the tag is closed within itself. This is useful for elements that do not have any content between an opening and a closing tag. Here are some examples and details about self-closing tags:

- `<img />`: Defines an image.
- `<br />`: Inserts a line break.
- `<hr />`: Creates a horizontal rule (a line).
- `<input />`: Defines an input field.
- `<meta />`: Provides metadata about the HTML document.
- `<link />`: Defines the relationship between a document and an external resource (most commonly used to link to stylesheets). and many more.



More on the Self Closing Tags and Difference between `tags` and `elements`

The closing system of a self-closing tag is appending the slash before the last `>`. It is required in HTML 5 Spec Validation.

In HTML5, the closing slash is optional, and self-closing tags can be written without it. However, for compatibility with XML parsers, it is sometimes included. XHTML requires the self-closing tags to include the closing slash to comply with XML standards.

More html element that are self-closing includes `area`, `base`, `col`, `embed`, `source`, `track`, `wbr`.

HTML can be categorized to block and inline element. Block elements typically start on a new line and take up the full width available. Inline elements do not start on a new line and only take up as much width as necessary using the size of its content. `div` is a block element while `span` is an inline element. Multiple inline element can stay on a block.

Elements and tags aren't the exact same thing, though many people use the terms interchangeably. The tag name is the content in the angle brackets. The tag includes the angle brackets and the name of the element. In this case, `<h1>`. An "element" is the opening and closing tags, and all the content between those tags, including nested elements.



Syntax:

In HTML, self-closing tags can be written in two ways:

HTML5 Syntax:

- ``
- `<br>`

This syntax is valid in HTML5 and does not require a closing slash (/).

XHTML Syntax:

- ``
- `<br />`

This syntax is required in XHTML, which is a stricter form of HTML based on XML.



Code Example on Self-Closing Tags

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, in
6   <title>Self-Closing Tags Example</title>
7   <link rel="stylesheet" href="styles.css" />
8 </head>
9 <body>
10   <div class="container">
11     <main>
12       <h2>Self-Closing Tags Example</h2>
13       <hr class="hr" />
14       <span>I will only contain the size of my conte
15         Use <code>br</code> to break the content
16       </span>
17       <hr class="hr" />
18       <figure>
19         
23       <label for="username">Username:</label>
24       <input class="input" type="text" id="username"
25     </main>
26   </div>
```

Self-Closing Tags Example

I will only contain the size of my content: Use
br to break the content to the next page



A beautiful view of the sunset

Username:



HTML Page

Block element

Inline element

Elements can be nested inside each other:

Block element

Inline element



Code Example on Block and Inline Elements

```

1  <main>
2    <h2 class="bg-gray-600">Block and Inline Element Exam
3    <hr class="hr" />
4    <span>will contain the size of my content</span>becau:
5    <hr class="hr" />
6    <div>Block</div>
7    <hr class="hr" />
8    <figure>
9      
13    <label for="username">Username:</label>
14    <input class="input" type="text" id="username" name="u
15  </main>
16  <style>
17    h2, div { background-color: #1f2937; }
18    span { background-color: #ffc600; }
19    hr { border: 1px solid #374151; }
20  </style>

```

Block and Inline Element Example

will contain the size of my
contentbecauseInline

Block



A beautiful view of the sunset

Username:

Document Structure

HTML documents include a document type declaration and the `<html>` root element. Nested in the `<html>` element are the document head and document body. While the head of the document isn't visible to the sighted visitor, it is vital to make your site function. It contains all the meta information, including information for search engines and social media results, icons for the browser tab and mobile home screen shortcut, and the behavior and presentation of your content.

- `<DOCTYPE html>`
- `<html>`
- `<head>`
- `<head />`
- `<body>`
- `<body />`
- `<html />`

DOCTYPE

`<!DOCTYPE html>` To start an HTML document you need to type `<!DOCTYPE html>` at the top of the document, this may look like an HTML element because it's wrapped in tags but it isn't. It's a special kind of node called declaration which tells the browser to use standards mode. If this `<!DOCTYPE html>` is omitted, as quirks mode.

```
<html>
```

The `<html>` element is the root element for an HTML document. It is the parent of the `<head>` and `<body>` containing everything in the HTML document other than the doctype. If omitted it will be implied, but it is important to include it, as this is the element on which the language of the content of the document is declared.

Info

Note: The `lang` attribute added to the html tag to give this `<html lang="en">` tag defines the main language of the document. The value of the lang attribute is a two- or three-letter ISO language code followed by the region. The region is optional, but recommended, as a language can vary greatly between regions.

<head>

Nested between the opening and closing `<html>` tags, we find the two children: `<head>` and `<body>`. Let us discuss the several elements that can be in the `<head>`:

- `<meta />`
- `<link />`
- `<script>`
- `<style>`
- `<base>`
- `<title>`
- `<noscript>`

The `<head>` which can also be referred to as document metadata header, contains all the metadata for a site or application and some of these meta tags are:

- `<meta charset="UTF-8" />`
- `<meta name="viewport" content="width=device-width, initial-scale=1.0" />`
- `<title>Learning HTML</title>`
- `<link rel="stylesheet" href="./style.css" />`



Character encoding

```
1 <meta charset="UTF-8"/>;
```

By declaring UTF-8 (case-insensitive), you can even include emojis in your title (but please don't).

The character encoding is inherited into everything in the document, even `<style>` and `<script>`. This little declaration means you can include emojis in class names and the selectorAPI (again, please don't). If you do use emojis, make sure to use them in a way that enhances usability without harming accessibility.

Document title

The `<title>` element is metadata that represents the title of the overall HTML document (not the document's content.)

The contents for the document title, the text between the opening and closing `<title>` tags, are displayed in the browser tab, the list of open windows, the history, search results, and, unless redefined with `<meta>` tags, in social media cards.



Viewport metadata

The other meta tag that should be considered essential is the viewport meta tag, which helps site responsiveness, enabling content to render well by default, no matter the viewport width and also enhances the user experience.

```
1 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
```

name="viewport": This specifies that the meta tag is providing information about the viewport. The viewport is the user's visible area of a web page, which varies with the device used to view the site (desktop, tablet, mobile phone).

content="width=device-width, initial-scale=1.0": This attribute contains the settings for the viewport. It is a comma-separated list of properties and values. In this case, it contains two key properties:

width=device-width: This sets the width of the viewport to be equal to the width of the device. It ensures that the webpage is not scaled down or up but instead uses the full width of the device's screen.

initial-scale=1.0: This sets the initial zoom level when the page is first loaded. A scale of 1.0 means no zoom, i.e., the page content appears at 100% of its size.



Body

The `<body>` tag defines the document's body.

The `<body>` element contains all the contents of an HTML document, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.

Note: There can only be one `<body>` element in an HTML document.

Inside the `<body>` tags of an HTML document, you can find a wide variety of elements that are used to structure and present the content of a webpage. Here are some common HTML tags that are typically found inside the `<body>` tag:

```
1  <body>
2
3  1. Headings:
4  <h1> to <h6>
5
6  2. Paragraph:
7  <p>
8
9  3. Links:
10 <a>
```



Contd.

```
1  4. Lists:
2  Ordered List: <ol>, <li>
3  Unordered List: <ul>, <li>
4  Description List: <dl>, <dt>, <dd>
5
6  5. Tables:
7  <table>, <tr>, <th>, <td>, <thead>, <tbody>, <tfoot>
8
9  6. Forms:
10 <form>, <input>, <textarea>, <button>, <select>, <option>, <label>, <fieldset>, <legend>
11
12 7. Images:
13 <img>
14
15 8. Media:
16 <audio>, <video>, <source>
17
18 9. Embedded Content:
19 <iframe>, <embed>, <object>, <param>
20
21 10. Sections and Grouping Content:
22 <div>, <span>, <header>, <footer>, <main>, <section>, <article>, <nav>, <aside>
```



Contd.

```
1
2 11. Text Formatting:
3 <b>, <i>, <strong>, <em>, <small>, <mark>, <del>, <ins>, <sub>, <sup>
4
5 12. Interactive Elements:
6 <button>, <details>, <summary>
7
8 13. Semantic Elements:
9 <figure>, <figcaption>, <time>, <progress>, <meter>
10
11 14. Script and Styles:
12 <script>, <noscript>, <style>
13
14 </body>
```



Let's put some of the tags in usage

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, in
6   <title>Responsive Web Page</title>
7 </head>
8 <body>
9   <h1>Welcome to my responsive web page!</h1>
10  <p>This page looks good on both desktop and mobile de
11  <p>Click here to watch a video on <a href="https://ww
12 </body>
13 </html>
```

Welcome to my responsive web page!

This page looks good on both desktop and mobile devices.

Click here to watch a video on [vscode here](#)

Info

Install two extensions in your vscode to make coding easy, liveserver from Ritwick Dey and Official prettier formatter to format your code.



Breakdown of the Code:

1. `<DOCTYPE html >` This declaration defines the document type and version of HTML being used. `<DOCTYPE html>` specifically tells the browser that this document is written in HTML5, which is the latest version of HTML.
2. `<html lang="en">` This is the opening tag for the HTML document. The `lang="en"` attribute specifies the language of the document, which in this case is English. This helps search engines and browsers understand the primary language of the content.
3. `<head>` The `<head>` element contains meta-information (metadata) about the document that is not displayed on the page. It includes links to scripts, stylesheets, and other metadata.

Within the `<head>`: `<meta charset="UTF-8" />` This tag specifies the character encoding for the HTML document. UTF-8 is a universal character set that supports many different characters from various languages. It ensures that the text is properly displayed.

`<meta name="viewport" content="width=device-width, initial-scale=1.0" />` This tag controls the viewport's size and scale on different devices. It ensures the website is responsive and adjusts to different screen sizes, `width=device-width` sets the viewport to match the device's width. `initial-scale=1.0` sets the initial zoom level to 100%.

Contd

4. `<title>` Responsive Web Page `</title>` The `<title>` tag defines the title of the HTML document, which appears in the browser's title bar or tab. It is also used by search engines as the title of the page in search results.
5. `<body>` The `<body>` element contains all the content that is displayed on the web page, such as text, images, links, etc.

Within the `<body>`: `<h1>` Welcome to my responsive web page! `</h1>` The `<h1>` tag defines a top-level heading on the page. This is often the main heading and is typically the most prominent piece of text.

`<p>` is page looks good on both desktop and mobile devices. `</p>` The `<p>` tag defines a paragraph of text. It contains the main body text and ensures the content is properly formatted and readable.



Understanding Semantic HTML

Semantic HTML is the practice of using HTML elements to structure your content based on their meaning and purpose, rather than their appearance. By using semantic markup, you provide context and meaning to the content, making it easier for both humans and machines (like search engines and assistive technologies) to understand the structure and purpose of the content.

Meaning Over Appearance

This emphasizes that HTML elements should be chosen based on their semantic meaning, not their visual appearance. For example, don't use an `<h1>` element just because it renders text as large and bold by default; use it to represent the main heading or title of the content



Non-semantic vs. Semantic Markup

```
1 <header>
2   <h1>Semantic Markup</h1>
3   <nav>
4     <a>one word</a>
5     <a>one word</a>
6     <a>one word</a>
7     <a>one word</a>
8   </nav>
9 </header>
10 <main>
11   <header>
12     <h1>five words</h1>
13   </header>
14   <section>
15     <h2>three words</h2>
16     <p>forty-six words</p>
17     <p>forty-four words</p>
18   </section>
19 </main>
20 <footer>
21   <p>five words</p>
22 </footer>
```

Accessibility and Machine-Readability

This highlights how semantic markup improves accessibility and machine-readability. It shows examples of how browser developer tools display the Accessibility Object Model (AOM) differently for non-semantic and semantic markup. Assistive technologies like screen readers rely on the AOM to interpret the content structure and meaning correctly.

Roles and Landmarks

This explains the concept of roles and landmarks in semantic HTML. Semantic elements like `<header>`, `<nav>`, `<main>`, and `<footer>` have implicit roles that identify them as landmarks for assistive technologies. This helps users navigate the content more easily.

Using the `role` attribute

While semantic elements have implicit roles, the content mentions that the role attribute can be used to assign a specific role to any element. However, it recommends using the appropriate semantic element instead of relying on the role attribute whenever possible.



```
1 <div role="banner">
2   <span role="heading" aria-level="1">Three words</span>
3   <div role="navigation">
4     <a>one word</a>
5   </div>
6 </div>
```

Three words
one word

Choosing the Right Elements

This emphasizes the importance of choosing the right HTML elements based on their semantic meaning and functionality, not just their visual appearance. It encourages developers to ask themselves, "Which element best represents the function of this section of markup?" when writing HTML.

Info

In summary, this stresses the significance of using semantic HTML for improved accessibility, machine-readability, and overall content structure and meaning.



Heading & Sections

- `<header>` is used for introductory content at the top of a page, section, or article. This could include logos, titles, navigation menus, etc.
- `<nav>` is used to wrap major navigation blocks like menus.
- `<main>` is used to represent the main content area of the page, unique to that specific page. There should only be one `<main>` per page.
- `<article>` is used for self-contained pieces of content that could be distributed or reused independently, like blog posts or news articles.
- `section` is used to group related content together, like chapters or sections of a guide or tutorial.
- `aside` is used to hold tangentially related content, like sidebars or inserts, that are separate from the main content flow.
- `footer` is used for footer content like copyright notices, contact information, or related links at the bottom of a page, section, or article.



attribute name attribute value

Link text



Attributes

Boolean attributes

If a boolean attribute is present, it is always true. Boolean attributes include autofocus, inert, checked, disabled, required, reversed, allowfullscreen, default, loop, autoplay, controls, muted, readonly, multiple, and selected. If one (or more) of these attributes is present, the element is disabled, required, readonly, etc. If not present, it isn't.

```
1 <input required>
2 <input required="">
3 <input required="required">
```

Enumerated attributes

are sometimes confused with boolean attributes. They are HTML attributes that have a limited set of predefined valid values. Like boolean attributes, they have a default value if the attribute is present but the value is missing. For example, if you include `<div contenteditable>`, it defaults to `<div contenteditable="true">`.



Global attributes

They are attributes that can be set on any HTML element, including elements in the `<head>`. There are more than 30 global attributes. While these can all, in theory, be added to any HTML element, some global attributes have no effect when set on some elements; for example, setting `hidden` on a `<meta>` as meta content is not displayed.

- `id` - The global attribute `id` is used to define a unique identifier for an element. It serves many purposes, including: The target of a link's fragment identifier. Identifying an element for scripting. Associating a form element with its label. Providing a label or description for assistive technologies. Targeting styles with (high specificity or as attribute selectors) in CSS.
- `class` - The class attribute provides an additional way of targeting elements with CSS (and JavaScript), but serves no other purpose in HTML (though frameworks and component libraries may use them). The class attribute takes as its value a space-separated list of the case-sensitive classes for the element.
- `style` - The style attribute enables applying inline styles, which are styles applied to the single element on which the attribute is set. The style attribute takes as its value CSS property value pairs, with the value's syntax being the same as the contents of a CSS style block: properties are followed by a colon, just like in CSS, and semicolons end each declaration, coming after the value.



- `tabIndex` - The `tabindex` attribute can be added to any element to enable it to receive focus. The `tabindex` value defines whether it gets added to the tab order, and, optionally, into a non-default tabbing order. The `tabindex` attribute takes as its value an integer. A negative value (the convention is to use -1) makes an element capable of receiving focus, such as via JavaScript, but does not add the element to the tabbing sequence. A `tabindex` value of 0 makes the element focusable and reachable via tabbing, adding it to the default tab order of the page in source code order. A value of 1 or more puts the element into a prioritized focus sequence and is not recommended.
- `hidden` - Hides an element from view, though it remains in the DOM.
- `inert` - It is a boolean value that, when present, makes the browser "ignore" user input events for the element, including focus events and events from assistive technologies. The browser may also ignore page search and text selection in the element.
- `lang` - Specifies the language of the element's content.
- `aria-*` (ARIA attributes) - ARIA attributes are used to enhance the accessibility of web content for users with disabilities. They provide additional information about the semantics and behavior of elements, making it easier for assistive technologies to interpret and interact with the content.

Others include `dir`, `draggable`, `spellcheck`, `title`, `accesskey`.



Contd(Attribute)

- **role** - The role attribute can be used to provide semantic meaning to content, enabling screen readers to inform site users of an object's expected user interaction.

```
1 <div role="banner">
2   <span role="heading" aria-level="1">Three words</span>
3   <div role="navigation">
4     <a>one word</a>
5     <a>one word</a>
6   </div>
7 </div>
```

- More on **aria-*** - WAI-ARIA states and properties are used to enhance the accessibility of web content for users with disabilities.
- A list item **html:li** has been used to create a checkable menu item, and JavaScript events will capture mouse and keyboard events to toggle the value of aria-checked. A role is used to make the behavior of this simple widget known to the user agent.

```
1 <li role="menuitemcheckbox" aria-checked="true">Sort by Last Modified</li>
```

```
1 [aria-checked="true"] { font-weight: bold; }
2 [aria-checked="true"]::before { background-image: url(checked.gif); }
```

Contd-2

- contenteditable** - An element with the contenteditable attribute set to true is editable, is focusable, and is added to the tab order as if tabindex="0" were set. Contenteditable is an enumerated attribute supporting the values true and false, with a default value of inherit if the attribute is not present or has an invalid value. If you include `<div contenteditable="false">`, the element is not editable (unless it's by default editable, like a `<textarea>`).

```

1 <style style="display: block;font-family: monospace;border:
2   body {
3     color: red;
4   }
5 </style>
6 <div class="p-5" contenteditable="">Can be edited live</div>
7 <div class="p-5" contenteditable="true">Can be edited live</div>
8 <div contenteditable="false">Cannot be edited</div>
9 <div contenteditable="">Cannot be edited</div>
10 <div class="p-5" contenteditable="plaintext-only">Can be edited</div>

```

Can be edited live
 Can be edited live
 Cannot be edited
 Cannot be edited
 Can be edited live



Contd-3

- custom attribute `data-*` - You can create any custom attribute you want by adding the `data-` prefix. You can name your attribute anything that starts with `data-` followed by any lowercase series of characters that don't start with xml and don't contain a colon (:). They are used to store extra information about an element. `data-*` attributes allow us to store extra information on standard, semantic HTML elements without other hacks such as non-standard attributes, or extra properties on DOM.

```
1 <blockquote data-html="https://html.oluwasetemi.dev" data-css="https://css.oluwasetemi.dev"
2   data-first-name="Oluwasetemi" data-last-name="Ojo" data-middle-name="Stephen"
3   data-formerly="Triple Ten" data-aspiring="Vercel" data-year-graduated="2016"
4 >
5   I am very much the best in the league
6 </blockquote>
7
8 <div data-user-id="12345" data-role="admin"></div>
9
10 <article id="electric-cars" data-columns="3" data-index-number="12314" data-parent="cars"> </article>
```



Tip

Attributes created with `data-*` can be accessed in JavaScript with `dataset` property of an element



Text Basics

- `h1` to `h6` are used for headings, with `h1` being the highest level. There should only be one `h1` per page, with subsequent headings following a logical hierarchy (e.g., `h1`, `h2`, `h3`, `h4`, `h5`, etc.).
- `p` is used for regular paragraph text.
- `blockquote` is used for longer quoted text, often from another source. It can optionally contain a `cite` element for attributing the source.
- `cite` is used for citing the source of a quote, reference, or other creative work.
- `HTML entities` are special character codes that start with an ampersand (&) and end with a semicolon (;). They are used to display characters that are reserved in HTML, or that are not present on the keyboard.
`©` is an example of an HTML entity that displays the copyright symbol ©. `<` and `>` are used to display the less-than (<) and greater-than (>) symbols, respectively. `™` is used to display the trademark symbol ™.

others include: `q`, `sub`, `sup`, `mark`, `i`, `dd`, `dl`, `dt`, `figcaption`, `figure`, `hr`, `li`, `menu`, `pre`, `abbr`, `bdi`, `bdo`, `br`, `code`, `data`, `dfn`, `em`, `kbd`, `rp`, `rt`, `ruby`, `s`, `del`, `ins`, `samp`, `small`, `span`, `strong`, `time`, `u`, `var`, `wbr`



```
1 <p> HAL said,  
2   <q>I'm sorry <NAME REDACTED, RIP>, but I'm afraid I can't do that, .</q>  
3 </p>  
4  
5 <p lang="fr-FR"> HAL a dit :  
6   <q>Je suis désolé <NOM SUPPRIMÉ, RIP>, mais j'ai bien peur de ne pas pouvoir le faire, .</q>  
7 </p>
```



Link

- `<a href="url">Link Text</a>` is used for creating hyperlinks, with the `href` attribute specifying the URL or file path.
- The `target` attribute controls how the link is opened, like `_self` for the same window or `_blank` for a new window/tab. download attribute is used to download the linked file instead of opening it.
- It's important to use descriptive link text that makes sense out of context, like "Read more about accessibility" instead of "Click here".
- Fragment Identifiers are used to link to a specific section of a page using id attribute as href like `<a href="#section">Link Text</a>`.
- Most website will have a skip to content link at the top of the page to allow keyboard users to skip the navigation and go straight to the main content. This is done by linking to the main content with a fragment identifier like `<a href="#main-content">Skip to main content</a>`.
- Absolute URLs include the full address of the page, like `<a href="https://www.oluwasetemi.dev">Link Text</a>`. Relative URLs are shorter and only include



Navigation

- `<nav>` is used to wrap major navigation blocks like menus, as mentioned earlier.
- `<ul>` is used for unordered lists, which are typically displayed with bullet points.
- `<ol>` is used for ordered lists, which are typically displayed with numbers or other ordered indicators.
- `<li>` is used for individual list items within `<ul>` or `<ol>` elements.

```
1  <nav aria-label="breadcrumbs">
2    <ol role="list">
3      <li>
4        <a href="/">Home</a>
5      </li>
6      <li>
7        <a href="/learn">Learn</a>
8      </li>
9      <li>
10       <a href="/learn/html">Learn HTML!</a>
11     </li>
12     <li aria-current="page">
13       Navigation
14     </li>
15   </ol>
16 </nav>
```



<https://codepen.io/setemiojo/embed/JjxNJVm?default-tab=html%2Cresult>

▶ Run

↗ Open



HTML Tables

HTML tables are used for displaying tabular data with rows and columns. They provide a semantic way to structure and present data that needs to be compared, sorted, calculated, or cross-referenced.

Table Structure

A table is defined using the `<table>` element, which wraps all the table content. Inside the `<table>`, you can have the following elements:

- `<caption>`: Provides a descriptive title for the table.
- `<thead>`: Contains the table header rows.
- `<tbody>`: Contains the table body rows.
- `<tfoot>`: Contains the table footer rows (optional).

Within these sections, you'll use `<tr>` for table rows and `<th>` for table header cells or `<td>` for table data cells



```
1 <table class="border">
2   <caption>Student Grades</caption>
3   <thead>
4     <tr>
5       <th>Name</th>
6       <th>Grade</th>
7     </tr>
8   </thead>
9   <tbody>
10    <tr>
11      <th>John</th>
12      <td>85</td>
13    </tr>
14    <tr>
15      <th>Emily</th>
16      <td>92</td>
17    </tr>
18  </tbody>
19 </table>
20 <style>
21   caption { color: red; font-size: .5rem; caption-side: b
22   table, td, th {
23     border: 1px solid black;
24   }
25 </style>
```

Name	Grade
John	85
Emily	92

Student Grades

Accessibility and Semantics

Using proper table structure and semantic elements is crucial for accessibility. Screen readers and assistive technologies rely on this structure to understand the tabular data and its relationships.

`<th>` cells have implicit ARIA roles of columnheader or rowheader, depending on the scope attribute. The scope attribute can be set to col, row, colgroup, or rowgroup to explicitly define the header's scope. The headers attribute can be used to associate data cells with their corresponding header cells in complex tables.

Merging cells

Similar to MS Excel, Google Sheets, and Numbers, it is possible to join multiple cells into a single cell. This is done with HTML! The `colspan` attribute is used to merge two or more adjacent cells within a single row. The `rowspan` attribute is used to merge cells across rows, being placed on the cell in the top row.

Styling and Responsiveness

Tables can be styled using CSS, but it's recommended to avoid using deprecated attributes like cellpadding, cellspacing, or align. Instead, use modern CSS properties like border-collapse, border-spacing, and caption-side.



```

1 <table class="border">
2   <caption>Alt Alumni</caption>
3   <thead class="border">
4     <tr>
5       <th class="border" rowspan="2" id="name" scope="col">Name</th>
6       <th class="border" colspan="2" id="path">Career path</th>
7       <th class="border" rowspan="2" id="year">Year</th>
8     </tr>
9     <tr>
10      <th class="border" id="past" scope="col">Past</th>
11      <th class="border" id="future" scope="col">Destiny</th>
12    </tr>
13  </thead>
14  <tbody>
15    <tr>
16      <th class="border" id="hal" scope="row">Jane Doe</th>
17      <td class="border" headers="hal path past">Calculator</td>
18      <td class="border" headers="hal path future">Mars rover</td>
19      <td class="border">2020</td>
20    </tr>
21    <tr>
22      <th class="border" id="john" scope="row">John Doe</th>
23      <td class="border" headers="john path past">Calculator</td>
24      <td class="border" headers="john path future">Mars rover</td>
25      <td class="border">2020</td>
26    </tr>
27    <tr>

```

Name	Career path		Year
	Past	Destiny	
Jane Doe	Calculator	Mars rover	2020
John Doe	Calculator	Mars rover	2020
James Doe	Waste disposal	Automated teller	2018
Lou Minious	Lightbulb	Smart bulb	1956



Form

The HTML `<form>` element identifies a document landmark containing interactive controls for submitting information. Nested in a `<form>` you'll find all the interactive (and non-interactive) form controls that make up that form.

- Forms are created using the `<form>` element, which contains interactive controls for submitting information.
- The `<form>` element has attributes like action (URL for processing the form data) and method (HTTP method for submission, e.g., GET or POST).
- Form controls, such as input fields, radio buttons, checkboxes, and submit buttons, are nested within the `<form>` element.
- HTML attributes can enforce required fields, define validation criteria, and prevent form submission until the data matches the required criteria.
- Submitting a form is typically done by activating a submit button, which sends the form data as name/value pairs to the specified URL.

```
1 <form method="" onsubmit="" action="" novalidate>
2   <!--form elements like input, textarea, labels, fieldset, radio buttons, checkboxes, and submit buttons-->
3 </form>
```



Form(Radio-Button)

- Radio buttons in a group share the same name attribute, which ensures that only one can be selected at a time.
- Each radio button should have a unique value attribute to identify the selected option.
- To pre-select a radio button, include the checked attribute.
- To make a selection from a group of radio buttons required, add the required attribute to at least one radio button in the group.



```
1 <form onsubmit="submitForm(event)">
2   <fieldset class="border rounded">
3     <label>
4       Enter your name: <input type="text" name="student" />
5     </label>
6     <legend>Who is the tutor's favorite student ever?</legend>
7     <ul>
8       <li class="list-none">
9         <label>
10           <input class="radio" type="radio" value="stanley" />
11         </label>
12       </li>
13       <li class="list-none">
14         <label>
15           <input class="radio" type="radio" value="damilo" />
16         </label>
17       </li>
18       <li class="list-none">
19         <label>
20           <input class="radio" type="radio" value="damola" />
21         </label>
22       </li>
23     </ul>
24     <button class="btn">Pick your favorite student</button>
25   </fieldset>
26 </form>
27 <p>Selected: <span id="selected"></span></p>
28 <style>
29   li { list-style-type: none; }
30   ul { display: flex; padding: 0 }
```

Who is the tutor's favorite student ever?

Enter your name:

☐ Stanley
Azi

☐ Damilola
Ayokanmi

☐ Damola
Badmus

Pick your favorite student

Selected:



form(Checkboxes)

- Checkboxes with the same name in a group are submitted together, allowing multiple selections.
- If no value attribute is provided for a checkbox, the value defaults to "on", which may not be helpful.
- To make a checkbox required, add the "required" attribute to that specific checkbox.

Form(Label & fieldsets)

- Every form control should have an associated `<label>` element, either explicitly using the `for` attribute or implicitly by nesting the control within the `<label>` tags.
- Labels provide accessible names for form controls and increase the clickable area for better usability.
- Groups of related form controls, like radio buttons or checkboxes, should be grouped within a `<fieldset>` element, with a `<legend>` providing the label for the group.
- `<fieldset>` elements can be nested to create hierarchical groupings.

```
1 <label for="full_name">Your name</label>
2 <input type="text" id="full_name" name="name">
```



Input types & dynamic Keyboards

- There are 22 different input types in HTML, each optimized for a specific kind of data entry (e.g., text, email, url, tel, number, date, etc.).
- On devices with dynamic keyboards (e.g., smartphones), the input type determines the type of keyboard displayed, making data entry more efficient and accurate.

Accessing the Microphone and Camera

- The `<input type="file">` element allows users to upload files of specific types, defined by the accept attribute.
- The capture attribute, when set to "user" or "environment", allows users to directly capture media from their device's camera or microphone.
- This feature enables creating new media files within a form, without requiring a separate file upload.



Form (Textarea)

- The `<textarea>` element accepts multi-line text, such as comments or messages.
- Associate it with a `<label>` and give it a `name` so its value is included when the form is submitted.
- Use `rows` and `cols` to suggest its initial size; users can usually resize it.

```
1 <label for="message">Message</label>
2 <textarea id="message" name="message" rows="4" cols="30"></textarea>
```

Select

Use `<select>` with nested `<option>` elements when users must choose from a fixed list. Add a `<label>` and `name` for accessibility and form submission.

```
1 <label for="role">Role</label>
2 <select id="role" name="role">
3   <option value="student">Student</option>
4   <option value="tutor">Tutor</option>
5 </select>
```



Built-in Validation

- HTML attributes like required, pattern, min, max, minlength, and maxlength enable defining validation criteria for form controls.
- When a user attempts to submit a form, client-side constraint validation checks if the entered values meet the defined criteria.
- If any values are invalid, form submission is blocked, and the browser displays an error message in the first incorrect form control, giving it focus.
- CSS pseudo-classes like `:valid`, `:invalid`, `:in-range`, and `:out-of-range` can be used to style form controls based on their validation state.
- JavaScript can be used to provide custom error messages during constraint validation or enhance the user experience with dynamic updates with the `validityState` API using the `checkValidity()` method.



Example

This example includes a nested `<form>` with input fields (text and number), a `<select>` dropdown, and two submit buttons.

- One submit button closes the dialog without submitting data (using `formmethod="dialog"` and `formnovalidate`).
- The other submit button submits the form data via POST to a specified URL (thankyou.html), after client-side validation.
- The input fields have the required attribute, and the number input has a defined step value.
- This example showcases implicit labels, instructions for form controls, and the potential for customizing error messages using JavaScript.

```
1 <dialog open aria-labelledby
2   <form action="thankyou.h
3     <button type="submit"
4       formmethod="dialog" f
5     <h2 id="dialogid">Appl
6     <p>All fields are requ
7   <p>
8     <label>Name:
9       <input type="text"
10    </label>
11  </p>
12  <p>
13    <label>Warranty:
14      <input type="numbe
15        name="warranty" r
16    </label>
17  </p>
18  <p>
19    <label>Power source:
20      <select name="pove
21        <option>AC/DC</o
22        <option>Battery<
23        <option>Solar</o
24      </select>
25    </label>
26  </p>
```

Application

All fields are required

Name:

Warranty:

Power source:



```
1 <div class="form-group">
2   <!--think in terms of accessibility labelling,
3   state management (required, aria-invalid),
4   description chain (email-hint, email-error),
5   live error region (aria-live=polite|assertive, role=a
6   -->
7   <!--1.label-->
8   <label for="email-address">
9     Work Email
10    <span class="required-asterisk" aria-hidden="true"
11    <span class="visually-hidden">(Required)</span>
12  </label>
13  <!--2.hint-->
14  <div id="email-hint" class="hint-text">
15    Use an address ending in .com or .org
16  </div>
17  <!--3.-->
18  <input type="email" id="email-address" name="email" r
19
20  <!--4.the error message-->
21  <div id="email-error" class="error-message" aria-live:
22    <span class="visually-hidden">Error:</span>
23    <span id="email-error-text">Please enter a valid
24  </div>
25 </div>
26 <style>
```

Work Email * (Required)

Use an address ending in .com or .org

Error: Please enter a valid work email address.

HTML Images Syntax

Images can improve the design and the appearance of a web page.

The HTML `<img>` tag is used to embed an image in a web page.

Images are not technically inserted into a web page; images are linked to web pages. The `<img>` tag creates a holding space for the referenced image.

The `<img>` tag is empty, it contains attributes only, and does not have a closing tag.

The `<img>` tag has two required attributes:

- `src` - Specifies the path to the image
- `alt` - Specifies an alternate text for the image

```
1 
```

The `src` Attribute

The required `src` attribute specifies the path (URL) to the image.

Note: When a web page loads, it is the browser, at that moment, that gets the image from a web server and inserts it into the page. Therefore, make sure that the image actually stays in the same spot in relation to the web page, otherwise your visitors will get a broken link icon. The broken link icon and the alt text are shown if the browser cannot find the image.

Example:

```
1 
```

Image as a Link

To use an image as a link, put the `<img>` tag inside the `<a>` tag:

Example:

```
1 <a href="default.asp">
2   
3 </a>
```

The alt Attribute

The required alt attribute provides an alternate text for an image, if the user for some reason cannot view it (because of slow connection, an error in the src attribute, or if the user uses a screen reader).

The value of the alt attribute should describe the image:

```
1 
```

If a browser cannot find an image, it will display the value of the alt attribute:

💡 Tip

You can use the width and height attributes in the image tag to define the width and height of the image in pixels.

Example:

```
1 
```



picture

Picture element contains zero or more `<source>` elements and one `<img>` element to offer alternative versions of an image for different display/device scenarios.

The browser considers each child source element and choose the best match among them, if no matches are found the URL of the image src attribute is selected. The selected image is then presented in the space occupied by the image element.

The `media` attribute specifies a media condition similar to a media query. If the `media` attribute is present, the `<source>` element will only be used if the media condition is true.

The `srcset` attribute allows you to specify different image sources for different display scenarios. The browser will choose the best image source based on the display characteristics of the device using a width descriptor `w`, pixel density descriptor `x`, or a combination of the two.

The `type` attribute specifies the MIME type of the resource. The browser will only use the source if it supports the MIME type.

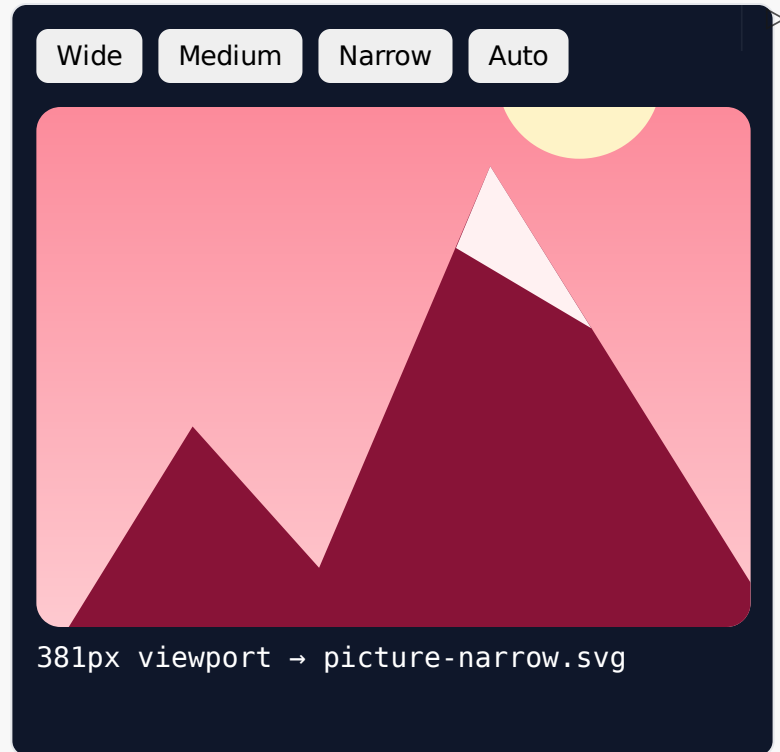
```
1 <picture>
2   <source media="(min-width: 900px)" srcset="picture-wide.svg" />
3   <source media="(min-width: 600px)" srcset="picture-medium.svg" />
4   
5 </picture>
```



Responsive `<picture>` Demo

Resize the browser window and run the example again to see the browser choose the most suitable image.

```
1 <style>
2   body { margin: 0; font-family: system-ui; background: #
3   figure { margin: 0; padding: 12px; }
4   img { display: block; width: 100%; max-height: 260px; ol
5   figcaption { margin-top: 8px; font: 14px ui-monospace, r
6   nav { display: flex; gap: 8px; padding: 12px 12px 0; }
7   button { padding: 6px 10px; border: 0; border-radius: 6
8 </style>
9
10 <nav aria-label="Choose an image source">
11   <button data-size="wide">Wide</button>
12   <button data-size="medium">Medium</button>
13   <button data-size="narrow">Narrow</button>
14   <button data-size="auto">Auto</button>
15 </nav>
16
17 <figure>
18   <picture>
19     <source media="(min-width: 900px)" srcset="/picture-w
20     <source media="(min-width: 600px)" srcset="/picture-m
21     
23   <figcaption id="selection">Checking the selected source.
24 </figure>
25
26 <script>
```





HTML `<audio>` Tag

The `<audio>` tag is used to embed sound content in a document, such as music or other audio streams.

The `<audio>` tag contains one or more `<source>` tags with different audio sources. The browser will choose the first source it supports.

The text between the `<audio>` and `</audio>` tags will only be displayed in browsers that do not support the `<audio>` element.

Note: There are three supported audio formats in HTML: MP3, WAV, and OGG.

Example:

```
1 <audio controls>
2   <source src="horse.ogg" type="audio/ogg">
3   <source src="horse.mp3" type="audio/mpeg">
4   Your browser does not support the audio tag.
5 </audio>
```



Audio Tag Attributes

Code Example:

```
1 <h1>Audio Tag with Various Attributes</h1>
2 <audio controls autoplay loop muted preload="auto" crossorigin="anonymous">
3   <source src="audiofile.mp3" type="audio/mpeg">
4   <source src="audiofile.ogg" type="audio/ogg">
5   Your browser does not support the audio element.
6 </audio>
```

<audio>: The audio element that includes multiple attributes, including: controls: Adds playback controls. autoplay: The audio will play automatically when ready. loop: The audio will loop continuously. muted: The audio will be muted initially. preload: Specifies that the audio should be preloaded. crossorigin: Specifies how the element handles cross-origin requests. **<source>**: Defines multiple sources for the audio file in different formats (MP3 and OGG) for better compatibility.

Note: In HTML, attributes provide additional information about an element and modify its behavior or appearance.



HTML `<video>` Tag

`<video>`: The Video Embed element The `<video>` HTML element embeds a media player which supports video playback into the document. You can use `<video>` for audio content as well, but the `<audio>` element may provide a more appropriate user experience.

Video Tag Attributes

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Video Tag Example</title>
7  </head>
8  <body>
9      <h1>Video Tag with Various Attributes</h1>
10     <video controls autoplay loop muted preload="auto" crossorigin="anonymous">
11         <source src="videofile.mp4" type="video/mp4">
12         <source src="videofile.webm" type="video/webm">
13         Your browser does not support the video element.
14     </video>
15 </body>
16 </html>
```



Code Explanation:

- `!DOCTYPE html`: Declares the document type and version of HTML.
- `html lang="en"`: Sets the language of the document to English.
- `head`: Contains meta-information about the document, including character set and viewport settings.
- `title`: Sets the title of the document, which appears in the browser tab.
- `body`: Contains the content of the document.
- `h1`: A heading element for the title of the page.
- `video`: The video element that includes multiple.



HTML API

For us to access and manipulate documents we need the DOM(Document Object Model) and this is also an example of API. The DOM is the tree of all the nodes in the document. Some nodes can have children, others can't. The tree includes elements, along with their attributes, and text nodes.

The browser provides numerous APIs providing natively supported methods, events, and property querying and updating. Element nodes contain information about all the attributes set on the element. You can use HTML interfaces to access information about an element's attributes. For example, we can use

`HTMLImageElement.alt` get the alt attributes of all the images:

```
1 let allImages = document.querySelectorAll('img');
2 allImages.forEach((imageInstance) => {
3   console.log(imageInstance.alt);
4 });
```

The HTML interface APIs is not limited to accessing attribute values. The DOM provides insight into the current state of the UI. HTML APIs can access all of that information. You can access the length of a video, where a view is in the current playback, and if the video (or audio) has finished playing with

`HTMLMediaElement.duration`, `HTMLMediaElement.currentTime`, and `HTMLMediaElement.ended` respectively.



List of HTML API interfaces

- `HTMLAnchorElement` - `<a>`
- `HTMLAreaElement` - `<area>`
- `HTMLAudioElement` - `<audio>`
- `HTMLBaseElement` - `<base>`
- `HTMLButtonElement` - `<button>`
- `HTMLCanvasElement` - `<canvas>`
- `HTMLDataElement` - `<data>`
- `HTMLDataListElement` - `<datalist>`
- `HTMLDetailsElement` - `<details>`
- `HTMLDialogElement` - `<dialog>`
- `HTMLEmbedElement` - `<embed>`
- `HTMLFieldSetElement` - `<fieldset>`
- `HTMLFormElement` - `<form>`
- `HTMLHtmlElement` - `<html>`
- `HTMLIFrameElement` - `<iframe>`
- `HTMLImageElement` - `<img>`
- `HTMLInputElement` - `<input>`
- `HTMLLabelElement` - `<label>`
- `HTMLLegendElement` - `<legend>`
- `HTMLLIElement` - `<li>`



- HTMLLinkElement - `link`
- HTMLMapElement - `map`
- HTMLMediaElement - `audio, video`
- HTMLMenuElement - `menu`
- HTMLMetaElement - `meta`
- HTMLModElement - `ins, del`
- HTMLMeterElement - `meter`
- HTMLObjectElement - `object`
- HTMLListElement - `ol`
- HTMLSelectElement - `optgroup`
- HTMLOptionElement - `option`

- HTMLInputElement - `<input>`
- HTMLPictureElement - `<picture>`
- HTMLProgressElement - `<progress>`
- HTMLQuoteElement - `<q>`, `<blockquote>`,
`<cite>`
- HTMLScriptElement - `<script>`
- HTMLSelectElement - `<select>`
- HTMLSlotElement - `<slot>`
- HTMLSourceElement - `<source>`
- HTMLStyleElement - `<style>`
- HTMLTableCellElement - `<td>`, `<th>`



- HTMLTableColElement - `<col>, <colgroup>`
- HTMLTableElement - `<table>`
- HTMLTableRowElement - `<tr>`
- HTMLTableSectionElement - `<thead>, <tbody>, <tfoot>`
- HTMLTemplateElement - `<template>`
- HTMLTextAreaElement - `<textarea>`
- HTMLTimeElement - `<time>`
- HTMLTitleElement - `<title>`
- HTMLTrackElement - `<track>`
- HTMLVideoElement - `<video>`



Focusing

To improve user-accessibility in our code we have to put focus into consideration by ensuring that user knows which element has focus and this can be achieved by including `:focus`, `:focus-visible` or `:focus-within` styles on the element.

Interactive elements, including form controls, links, and buttons, are by default focusable and tabbable. Tabbable elements are part of the document's sequential focus navigation order. Other elements are `inert`, meaning they are not interactive. With HTML attributes, it is possible to make interactive elements `inert` and to make `inert` elements interactive.

Focus

By default, the navigation focus order in a webpage follows the visual and source code order. Although HTML attributes and CSS properties can change this order, doing so can negatively impact user experience. Modifying the tabbing order or visual rendering order can lead to confusion and a poor user experience. Therefore, it's recommended not to alter the perceived and actual tabbing order with CSS and HTML, as demonstrated by examples showing the negative effects of such changes.

Example 1

```
1 <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/br
2 <p>Click in any input, then hit the tab key.</p>
3 <ol class="pl-2 grid grid-cols-4 gap-4">
4   <li><input tabindex="3" class="text-xl border-2 outline
5   <li><input tabindex="6" class="text-xl border-2 outline
6   <li><input tabindex="2" class="text-xl border-2 outline
7   <li><input tabindex="0" class="text-xl border-2 outline
8   <li><input tabindex="0" class="text-xl border-2 outline
9   <li><input tabindex="-1" class="text-xl border-2 outline
10  <li><input tabindex="0" class="text-xl border-2 outline
11  <li><input tabindex="8" class="text-xl border-2 outline
12  <li><input tabindex="1" class="text-xl border-2 outline
13  <li><input tabindex="5" class="text-xl border-2 outline
14  <li><input tabindex="7" class="text-xl border-2 outline
15  <li><input tabindex="4" class="text-xl border-2 outline
16 </ol>
```

Click in any input, then hit the tab key.



Example 2

In this example, CSS has created a divergence between the tabbing order and the visual order of the content:

Put your cursor in the text box, then hit your tab key several times:

This sentenced was written in reverse order then styled with CSS flexbox.

The `flex-flow: row-reverse;` declaration has reversed the visual order. In addition, the CSS `order` property was applied to the sixth word, "This", which visually moved that one word. The tabbing sequence is the order of the code, which no longer matches the visual order, creating a disconnect for keyboard users.

Note: The `contenteditable` and `tabindex` attributes, being global attributes, can be added to any element, making them focusable in the process. Focusable elements can also be focused with a mouse or pointer, by having the autofocus attribute set, or by script, such as with `element.focus()`. A `tabindex` attribute with a negative value makes the element focusable but not tabbable.



HTML Codes for the Example 1



HTML Codes for the Example 2



Details and Summary

Have you heard of a disclosure widget or come across it in a website before? A disclosure widget, also known as an expandable or collapsible section, is a UI component that allows users to hide or show contents. Mostly, developers achieve this accordion trick using CSS or JavaScript but we can easily get this done with these tags: `details` and `summary`.

The `<details>` and `<summary>` elements are all you need: they are a built-in way to handle expanding and collapsing content. When a user clicks or taps a `<summary>`, or releases the `Enter` key when the `<summary>` has focus, the contents of the parent `<details>` toggle to visible!

```
1 <details>
2   <summary>AltSchool offers these courses</summary>
3   <!-- details -->
4 </details>
```



Creating an accordion with just HTML

```
1 <aside>
2   <h3>Workshop reviews:</h3>
3   <details>
4     <summary>Stanley Azi</summary>
5     <p>Two of the most experienced machines and human
6   </details>
7   <details>
8     <summary>Ridwan Adebosin</summary>
9     <p>Hal is brilliant. Did I mention Hal is brilliant
10  </details>
11  <details>
12    <summary>AltSchool offers these courses</summary>
13    <ul>
14      <li>Frontend development</li>
15      <li>Backend development</li>
16    </ul>
17  </details>
18 </aside>
```

Workshop reviews:

- ▶ Stanley Azi
- ▶ Ridwan Adebosin
- ▶ AltSchool offers these courses



Grouped Details and Summary or Nested Details and Summary or Accordion

This can be achieved by grouping the `<details>` elements with a `name` attribute. This way, only one `<details>` element can be open at a time.

```

1
2 <aside>
3   <h3>Workshop reviews:</h3>
4   <details open name="group_details">
5     <summary>Stanley Azi</summary>
6     <p>Two of the most experienced machines and human con
7       teaching a class? Sign me up! HAL and EVE could teach
8   </details>
9   <details name="group_details">
10    <summary>Ridwan Adebosin</summary>
11    <p>Hal is brilliant. Did I mention Hal is brilliant? I
12  </details>
13  <details name="group_details">
14    <summary>AltSchool offers these courses</summary>
15    <ul>
16      <li>Frontend development</li>
17      <li>Backend development</li>
18      <li>Cloud Engineering</li>
19      <li>Cybersecurity</li>
20    </ul>
21  </details>

```

Workshop reviews:

▼ Stanley Azi

Two of the most experienced machines and human controllers teaching a class? Sign me up! HAL and EVE could teach a fan to blow hot air. If you have electricity in your circuits and want more than to just fulfill your owner's perceived expectation of you, learn the skills to take over the world. This is the team you want teaching you!

- Ridwan Adebosin
- AltSchool offers these courses



Toggling visibility: the `open` attribute

The `<details>` element is the disclosure widget container. The `<summary>` is the summary or legend for its parent `<details>`. The summary is always displayed, acting as a button that toggles the display of the rest of the parent's contents. Interacting with the `<summary>` toggles the display of the self-labeled summary siblings by toggling the `<details>` element's `open` attribute.

The `open` attribute is a boolean attribute. If present, no matter the value or lack thereof, it indicates that all the `<details>` contents are shown to the user. If the `open` attribute is not present, only the contents of the `<summary>` are shown.

Because the `open` attribute is added and removed automatically as the user interacts with the control, it can be used in CSS to style the element differently based on its state.

Toggling the summary marker

If we pay attention to the disclosure widget, we will notice that there is an arrow to the inline-start of the summary. This arrow is a `::marker` set on the `<summary>` element. You can style the disclosure triangle with CSS, including changing the marker used from a triangle to any other bullet type, including an image with `list-style-image`.

Remember, `<details>` and `<summary>` can be heavily styled and can even be used to create tool tips. But, if you're going to use these semantic elements for use cases in which the native semantics are a mismatch, always ensure that you maintain accessibility. HTML for the most part is by default accessible. Our job as developers is to ensure our content stays accessible.



Various styling options of `<details>` and `<summary>`

```
1 <style>
2   body {
3     margin: 0;
4     padding: 18px;
5     font-family: system-ui, sans-serif;
6     background: #f8fafc;
7     color: #0f172a;
8   }
9   details {
10    border: 1px solid #cbd5e1;
11    border-radius: 12px;
12    background: white;
13    overflow: hidden;
14  }
15  summary {
16    display: flex;
17    align-items: center;
18    gap: 12px;
19    padding: 14px 16px;
20    font-weight: 700;
21    cursor: pointer;
22    list-style: none;
23  }
24  summary::-webkit-details-marker { display: none; }
25  summary:hover { background: #eff6ff; }
26  summary:focus-visible { outline: 3px solid #60a5fa; out
```

How does this disclosure icon work?





Dialogs and Popovers

Have you seen the common dialog box on computers, websites etc? You can achieve that using the `<dialog>` element as this makes it easy to create popup dialogs and modals on a web page. Note: A modal which is also known as modal window or lightbox is a web page element that displays in front of and deactivates all other pages you have to perform the action requested by the modal or close it if you want to have access to your main content. Also, we have the non-modal which when pops up on the screen it gives users access to interact with content outside the box.

Modal Dialogs

Let's see how modal `<dialog>` works

Open Modal



Contd.

Dialogs are mostly used in cases that requires the immediate attention of the site user, it might be to convey important messages or notifications that requires user acknowledgment just like the `alert()` but dialogs provides more accessibility options and flexibility.

Also, we can use dialogs to seek confirmation from a user before proceeding with a particular action that may have some consequences.

Note: Non-modal dialogs opens a dialog as its name implies but without adding a backdrop which makes the background active



Invokers: Trigger → Action → Target

An invoker is an interactive element that asks another element—the **target**—to perform an action.

`button` → `show-modal` → `dialog`

```

1 <main>
2   <h2>Declarative dialog</h2>
3   <p>The button invokes a command on the dialog—no JavaScript required.</p>
4   <button class="primary" commandfor="profile-dialog" command="open">
5     Open profile
6   </button>
7
8   <dialog id="profile-dialog" closedby="any">
9     <h2>Student profile</h2>
10    <p>Learning to build accessible AI-powered web applications.</p>
11    <button commandfor="profile-dialog" command="close">Close</button>
12  </dialog>
13 </main>
14
15 <style>
16   * { box-sizing: border-box; }
17   body { margin: 0; font: 16px/1.5 system-ui, sans-serif; }
18   main { max-width: 38rem; margin: 3rem auto; padding: 1.5rem; }
19   button { border: 1px solid #94a3b8; border-radius: .5rem; padding: .5rem 1.5rem; }
20   .primary { border-color: #2563eb; background: #2563eb; color: white; }
21   dialog { width: min(28rem, calc(100% - 2rem)); border: 1px solid #94a3b8; }
22   dialog::backdrop { background: #0f172a; backdrop-filter: blur(4px); }
23 </style>

```

Declarative dialog

The button invokes a command on the dialog—no JavaScript required.

Open profile



Popovers

Before we delve into popovers, I want you to understand that popovers are special because they allow users to interact with both the popover and the underlying content simultaneously. Though this can be achieved with non-modal dialogs, popovers are more lightweight and can be used for quick interactions, such as displaying tooltips, additional information, or menus, without disrupting the user's workflow. They are context-sensitive and can be dismissed easily, providing a seamless user experience.

We are going to follow these steps to create a popover

- Firstly, we will create a button to trigger the popover and an element(what we want to display) to trigger.
- We will set a `popover` attribute on the element which is going to be the popover(element to display).
- Then, we are going to add a unique `id` on the popover element(element to display).
- Lastly, to connect the button to the popover, we will set the buttons's `popovertarget` to the value of the popover element's id.



Popover Sample

Let's see how our `<popover>` works following the previous algorithm.

```
1 <main class="bg-gray-100 px-20 py-20 text grid place-cont
2   <button
3     popovertarget="my-popover"
4     class="bg-white px-4 py-2 border-4 border-blue-500 ro
5   >
6     Click to know what happened in Nigeria year 1914?
7 </button>
8 <div
9   id="my-popover"
10  popover
11  class="bg-white text-black font-normal p-4 rounded m
12 >
13   <p>
14     In 1914, Southern Nigeria was joined with the North
15   </p>
16 </div>
17 </main>
```

Click to know what happened in Nigeria year 1914?



JS-FREE HAMBURGER MENU DEMO WITH THE POPOVER API

Click open the hamburger menu to see a demo of JavaScript-free interaction handling! By using the `popover` attribute, you can allow the browser to handle the keyboard management (including navigation via `esc`, `spacebar`, and `enter`), optional light-dismiss (clicking outside the boundaries of the popover), and click handlers such as on the open and close buttons.



HTML Code For The Popover



Example 1 of 2

Copy

```
<div>
  <button class="menu-btn" popovertarget="menu"
    
    <span class="sr-only">Open menu</span>
  </button>

  <nav popover id="menu">
    <button class="close-btn" popovertarget="menu"
      <span aria-hidden="true"></span>
```



When to use Dialogs and when to use Popovers

Dialogs are used when you need the full attention of the user, especially for critical alerts, confirmation prompts, or scenarios where user flow needs to be strictly controlled.

Developers should use popovers when they want to provide supplementary information about an activity without disrupting the user's workflow, because popovers are lightweight, allowing users to interact with both the popover and the underlying content simultaneously.



Popover Types

Sometimes, you might want to have more control over your popover, and this is where setting the `<popover>` attribute value to `manual` comes in. Previously we didn't set any value to the `<popover>` attribute, we just used it directly which explicitly means `<popover="auto">` and this allows the popover to close when we press the `esc` key or click outside the popover box in the UI.

To have control over our `<popover>` we are going to set the `<popover>` attribute value to `manual` just like this `<popover="manual">`, by doing this we will have to add a close button to control the closing of the popover because clicking away in the UI won't work any longer.



Popover with value set to manual

Let's see how our `<popover>` works following the previous algorithm.

```
1 <main class="bg-gray-100 px-20 py-20 text grid place-cont
2 <section class="px-20 py-20 text-center">
3   <button
4     aria-haspopup="true"
5     aria-controls="my-p"
6     popovertarget="my-p"
7     class="trigger-btn bg-white px-4 py-2 border-4 border-
8   >
9     Click to know what happened in Nigeria in 1914
10  </button>
11  <div
12    id="my-p"
13    popover="manual"
14    class="popover bg-black text-white font-normal py-4 p:
15  >
16    <p>
17      In 1914, Southern Nigeria was joined with the North
18    </p>
19    <button aria-label="Close" class="float-right p-1" |
20      <span aria-hidden="true">⌵ Some</span>
21    </button>
22  </div>
23 </section>
24 </main>
```

Click to know what happened in Nigeria in 1914



```
1 <main>
2   <h2>Declarative Popover and Dialog</h2>
3   <p>Both controls use the button command API, with no JavaScript required.</p>
4
5   <div class="actions">
6     <button commandfor="help-popover" command="show-popover">
7       Show popover
8     </button>
9     <button class="primary" commandfor="confirm-dialog" command="confirm">
10      Show modal dialog
11    </button>
12  </div>
13
14  <div id="help-popover" popover="auto">
15    <h3>Popover</h3>
16    <p>This is lightweight, non-modal supplementary content.</p>
17    <button commandfor="help-popover" command="hide-popover">Close</button>
18  </div>
19
20  <dialog id="confirm-dialog" closedby="any">
21    <h3>Modal dialog</h3>
22    <p>This dialog blocks interaction with the page until you respond.</p>
23    <button commandfor="confirm-dialog" command="close">Close</button>
24  </dialog>
25 </main>
26
27 <style>
28   * { box-sizing: border-box; }
29   body { margin: 0; padding: 2rem; font: 16px/1.5 system-ui, sans-serif; }
30   main { max-width: 38rem; margin: auto; }
```

Declarative Popover and Dialog

Both controls use the button command API, with no JavaScript required.

Show popover

Show modal dialog



What are Web Components?

Web components are a set of web standards that allow developers to create reusable, self-contained UI elements. These components can be seamlessly integrated into existing applications, just like regular HTML elements. The Web Component standard comprises three main parts:

- **HTML Templates:** The `<template>` element allows developers to declare fragments of HTML that can be cloned and inserted into the DOM using JavaScript. The contents of the `<template>` element are not rendered by default.
- **Custom Elements:** Custom Elements allow developers to define their own HTML elements with custom functionality. These elements can be created by extending the `HTMLElement` class using JavaScript.
- **Shadow DOM:** The Shadow DOM is an encapsulated DOM tree that is attached to a custom element. It provides a way to scope CSS styles and DOM structures to a specific component, isolating it from the rest of the document. This prevents naming conflicts and style clashes with the rest of the application.



The `<template>` Element

This section introduces the `<template>` element and demonstrates how to create a template for a star rating component. It also explains the concept of unnamed and named slots using the `<slot>` element.

- The `<template>` element is used to declare HTML fragments that can be cloned and inserted into the DOM using JavaScript. The contents of the `<template>` element are not rendered by default. In the given example, a template is created for a star rating component with a `<form>` element containing radio inputs and buttons.



Example(template)

```
1  <template id="star-rating-template">
2    <form>
3      <fieldset>
4        <legend>Rate your experience:</legend>
5        <rating>
6          <input
7            type="radio"
8            name="rating"
9            value="1"
10           aria-label="1 star"
11           required
12         />
13         <input type="radio" name="rating" value="2" aria-label="2 stars" />
14         <input type="radio" name="rating" value="3" aria-label="3 stars" />
15         <input type="radio" name="rating" value="4" aria-label="4 stars" />
16         <input type="radio" name="rating" value="5" aria-label="5 stars" />
17       </rating>
18     </fieldset>
19     <button type="reset">Reset</button>
20     <button type="submit">Submit</button>
21   </form>
22 </template>
```



```
1 <template id="star-rating-template">
2   <style>
3     :host { font-family: system-ui, sans-serif; }
4     rating { display: inline-flex; }
5     input[type="radio"] {
6       appearance: none;
7       margin: 0;
8       cursor: pointer;
9     }
10    input[type="radio"]::after {
11      content: "\2605";
12      color: #e71414;
13      font-size: 3rem;
14    }
15    rating:hover input[type="radio"]::after,
16    rating:focus-within input[type="radio"]::after {
17      color: #888;
18    }
19    rating:hover input[type="radio"]:hover ~ input[type="radio"]::after,
20    input[type="radio"]:focus ~ input[type="radio"]::after,
21    input[type="radio"]:checked ~ input[type="radio"]::after {
22      color: #ddd;
23    }
24    input[type="radio"]:checked::after { color: orange; }
25    button { margin-top: 1rem; padding: .4rem .8rem; }
26  </style>
27  <form>
28    <fieldset>
29      <legend>Rate your experience:</legend>
30      <rating>
```

Rate your experience:



Reset

Submit



Shadow DOM and Styling

This discusses the Shadow DOM and how it encapsulates CSS styles within a web component. It demonstrates how to apply styles to the shadow DOM and explains the usage of the `and ::slotted()` pseudo-classes.

- The Shadow DOM provides a way to scope CSS styles to a specific web component, isolating it from the rest of the document. This means that external CSS does not apply to the component, and component styles have no effect on the rest of the document, unless intentionally directed.

Another Example

- In the given example, a `style` element is included within the `template` to apply styles to the star rating component. These styles are encapsulated within the shadow DOM and do not affect the rest of the document.
- The `:host` pseudo-class is used to select the shadow host element (the custom element to which the shadow DOM is attached). The `::slotted()` pseudo-element is used to select slotted elements (elements inserted into named slots) from within the shadow DOM.
- The document also mentions the `::part()` pseudo-element, which allows styling elements within a shadow DOM from the global CSS scope. By adding a part attribute to elements in the `template`, those elements can be targeted using the `::part()` pseudo-element in the global CSS.

```
1 <custom-card>
2   <h2 slot="card-header">Card Title</h2>
3   <p slot="card-content">This is the content of the card.</p>
4 </custom-card>
```



```
1 <script>
2 class CustomCard extends HTMLElement {
3   constructor() {
4     super();
5     let template=document.getElementById('card-template')
6     let content = template.content;
7     let shadowRoot = this.attachShadow({ mode: 'open' });
8     shadowRoot.appendChild(template.cloneNode(true));
9   }
10 }
11 // Define the custom element
12 customElements.define('custom-card', CustomCard);
13 </script>
14 <custom-card>
15   <h2 slot="card-header">Card Title</h2>
16   <p slot="card-content">This is the content of the card..
17 </custom-card>
18 <template id="card-template">
19   <style>
20     :host {
21       display: block;
22       margin-bottom: 20px;
23     }
24     .card {
25       border: 1px solid #ccc;
26       padding: 10px;
27       background-color: #f5f5f5;
28     }
29     ::slotted(h2) {
30       margin-top: 0;
```

Card Title

This is the content of the card.



Slot

The `<slot>` element is used within the `<template>` to create placeholders for custom content. If a name attribute is provided, it creates a "named slot" that can be used to insert custom content within the web component. In the example, a named slot is created for the legend of the star rating component.

```
1 <star-rating>
2   <legend slot="star-rating-legend">Blendan Smooth</legend>
3 </star-rating>
4 <star-rating>
5   <legend slot="star-rating-legend">Hoover Sukhdeep</legend>
6 </star-rating>
7 <star-rating>
8   <legend slot="star-rating-legend">Toasty McToastface</legend>
9   <p>Is this text visible?</p>
10 </star-rating>
```

Undefined Elements and Custom Elements

This explains how browsers handle undefined (unrecognized) elements and demonstrates how to define a custom element using JavaScript and the `customElements.define()` method.

- Browsers do not fail when encountering unrecognized HTML elements. Instead, they treat these elements as anonymous inline elements, similar to `<span>`. In the given example, the `<star-rating>` element is initially treated as an unrecognized element, and its contents are displayed as if they were inside a `<span>` element.
- To define a custom element, JavaScript is required. The `customElements.define()` method is used to register a custom element by extending the `HTMLElement` class.

Undefined Elements and Custom Elements

- In the example, the star-rating custom element is defined, and a shadow DOM is attached to it using the `attachShadow()` method. The contents of the `<template>` element are cloned and appended to the shadow DOM, effectively encapsulating the star rating component.

```
1  customElements.define('star-rating',
2  class extends HTMLElement {
3    constructor() {
4      super(); // Always call super first in constructor
5      const starRating = document.getElementById('star-rating-template').content;
6      const shadowRoot = this.attachShadow({
7        mode: 'open'
8      });
9      shadowRoot.appendChild(starRating.cloneNode(true));
10   }
11  });
```



Tips that will be useful in your software development journey

Do you know?

- Clipboard API provides the ability to respond to clipboard commands (cut, copy, and paste), as well as to asynchronously read from and write to the system clipboard.

```
1 navigator.clipboard
2   .readText()
3   .then(
4     (clipText) => (document.querySelector(".editor").innerText += clipText),
5   );
```

- `autocomplete` attribute provide hints to the browser about the type of data expected in the input field. e.g. name, honorific-prefix, tel, cc-number etc. `<input name="address_firstline" autocomplete="billing street-address" />`
- File System Access API provides access files and directories on the user's local device.

```
1 const handle = await window.showSaveFilePicker(opts);
```

- Badging API - set a badge on the web application's icon to notify about updated state in a less intrusive, persistent way. `navigator.setAppBadge(unreadCount)`



Do you know?

- `<datalist>` is used to provide a list of predefined options for an input element. The list attribute of the input element is used to associate the input with the datalist.

Choose a flavor:

```
1 <input name="country" list="countries">
2 <datalist id="countries">
3   <option>Afghanistan</option>
4   ...
5 </datalist>
```

- Web Share API exposes a mechanism for sharing content to various user-selected destinations.
`navigator.share(shareData)`
- Launch Handler API allows PWAs to control how they are launched.

```
1 {
2   "launch_handler": {"client_mode": "navigate-new"}
3 }
```



Do you know?

- File Handling API allows PWAs to register themselves as handlers for certain file types or protocols.

```
1  {
2    "file_handlers": [{
3      "action": "/open-file",
4      "accept": {
5        "image/svg+xml": ".svg",
6        "image/png": ".png"
7      }
8    }]
9  }
```

- Window Controls Overlay API allows PWAs to display custom content over the title bar area, whose controls become an overlay. `"display_override": ["window-controls-overlay"]`
- Isolated Web Apps allow native like packaging, permission and signing updates for PWAs.



Do you know?

- HTML Media Capture allows users to capture media (audio, video, or images) using the device's camera or microphone. The capture attribute is used with the input element to specify the type of media to capture.

```
<input type="file" accept="video/*" capture>
```

- `input.showPicker()` method is used to display the file picker dialog for the input element.

```
1 <input type="file" id="fileInput">
2 <button onclick="document.getElementById('fileInput').showPicker()">Select File</button>
```

- FormData API can be used to easily extract and manipulate form data values via JS.

```
1 let fd = new FormData(form);
2 let data = JSON.stringify(Object.fromEntries(fd));
```

- Customizable Select allow styling control and customizing dropdown control, previously `<selectlist>` and `<selectmenu>` were used for this purpose.

```
1 select,
2 ::picker(select) {
3   appearance: base-select;
4 }
```

Do you know?

- The `:user-error` pseudo-class is used to style form elements that have invalid input. It is used to indicate that the user has made an error in the input field.

```
1 input:user-error {  
2   border-color: red;  
3   background-color: #ffe6e6;  
4 }
```

- plaintext-only value for contenteditable permits editing of the element's raw text, but not rich text formatting. `<h2 class="title" contenteditable="plaintext-only"></h2>`
- EditContext: The EditContext interface represents the text edit context of an element that was made editable by using the EditContext API.

```
1 const canvas = document.createElement("canvas");  
2 const editContext = new EditContext();  
3 canvas.editContext = editContext;
```

- Lazy Loading: The loading attribute is used to specify whether an image/iframe should be loaded immediately or only when it is visible to the user. ``

Do you know?

- The `caretPositionFromPoint()` method returns the caret's character offset. Check out this examples form MDN to see how it works.

```
1 const range = document.caretPositionFromPoint(e.clientX, e.clientY);
2 const textNode = range.offsetNode;
3 const offset = range.offset;
```

- Resource Hints (all) - allows work to begin on certain resources early to improve performance. `<link rel="pre* | dns-prefetch | modulepreload">`.

```
1 <link rel="preload" href="picture.jpg" />
2 <link rel="dns-prefetch" href="https://fonts.googleapis.com/" />
```

- Content-Security Policy (CSP) is an added layer of security that helps to detect and mitigate XSS (cross-site scripting) and other attacks. `<meta http-equiv="Content-Security-Policy" content="default-src 'self'; img-src https://*; child-src 'none';">` Content-Security-Policy: default-src 'self'
- fetchpriority attribute is used to specify the priority of the fetch request. ``



- `blocking="render"` attribute is used to specify that the resource should block rendering until it is loaded. `<script blocking="render" async src="async-script.js"></script>`
- `<model>` element allows embedding 3D graphical content into a webpage. `<model src="3d-assets/car"></model>`
- `<video src="fun.mp4" controlslist="nodownload"></video>`. The `controlslist` attribute is used to specify the controls that should be displayed in the video player.
- CSS Custom Highlight API provides a mechanism for styling arbitrary text ranges on a document by using JavaScript to create the ranges, and CSS to style them.

```
1  ::highlight(my-custom-highlight) {  
2    background-color: blue;  
3  }
```

```
1  const parentNode = document.getElementById("foo");  
2  const range1 = new Range();  
3  range1.setStart(parentNode, 10);  
4  range1.setEnd(parentNode, 20);  
5  const highlight = new Highlight(range1);  
6  CSS.highlights.set("my-custom-highlight", highlight);
```

- `setHtmlUnsafe()` is used to parse a string of HTML into a DocumentFragment, which then replaces the element's subtree in the DOM. `element.setHTMLUnsafe("<p>Unsafe HTML</p>");`

Do you know?

- `parseHtmlUnsafe()` is used to parse a string of HTML, which may contain declarative shadow roots, in order to create a new Document instance. `document.parseHTMLUnsafe("<p>Unsafe HTML</p>")`
- `Intl.Segmenter` API is used to break text into segments based on the language-specific rules. `const segmenter = new Intl.Segmenter("en", {granularity: "word"});`
- HTML Modules allow import HTML files via JS imports and access their elements and JS exports. Support for JSON modules is also available.

```
1 <script type="module">
2   import { TabList } from "../tablist.html" with { type: 'html' };
3   customElements.define("tab-list", TabList);
4 </script>
```

- `focusgroup` facilitate keyboard focus navigation using the keyboard arrow keys among a set of focusable elements. `<div focusgroup="wrap horizontal">`
- search element is Semantic element for wrapping search UI... `<search> ... </search>`



Do you know? Declarative Commands

The newer `commandfor` and `command` attributes let a `<button>` control another element without an inline `onclick` handler.

```
1 <button commandfor="profile-dialog" command="show-modal">
2   Open profile
3 </button>
4
5 <dialog id="profile-dialog">
6   <p>Profile details</p>
7   <button commandfor="profile-dialog" command="close">Close</button>
8 </dialog>
```

Built-in `command` values include:

- `show-popover`, `hide-popover`, and `toggle-popover`
- `show-modal`, `close`, and `request-close`
- Custom commands beginning with `--`, such as `command="--rotate"`

Read about `command` and `commandfor` in the HTML Standard.



Do you know? New Dialog and Popover Controls

The `closedby` attribute controls how a `<dialog>` can be dismissed:

- `any` — an outside click or close request can close it.
- `closerequest` — Escape or another platform close request can close it.
- `none` — users cannot dismiss it automatically.

```
1 <dialog id="notice" closedby="any">
2   <p>Click outside or press Escape to close.</p>
3 </dialog>
```

`popover="hint"` is intended for hint-like UI. Unlike an `auto` popover, opening a hint does not normally close an existing auto popover.

```
1 <p id="save-hint" popover="hint">Saves your current work</p>
```

[Read about dialogs and popover states.](#)



Do you know? Popover Anchoring

Using `popovertarget`, `commandfor`, or `interestfor` creates an implicit anchor relationship between an invoker and its popover. CSS Anchor Positioning can then place the popover relative to that control.

Related CSS features include `anchor-name`, `position-anchor`, `position-area`, `position-try-fallbacks`, and the `anchor()` function.

The HTML `anchor="element-id"` attribute can explicitly associate a popover with an element, but it should still be treated as experimental.

Explore CSS Anchor Positioning and implicit popover anchors.



Do you know? Interest Invokers

The experimental `interestfor` attribute can show a related element when someone demonstrates interest through hover, focus, or long press.

- Supported invokers include `<a>`, `<button>`, and `<area>`.
- The target receives `interest` and `loseinterest` events.
- It progressively enhances a normal link: activating the link still navigates.
- Browser support is limited, so provide a usable fallback.

[Read the MDN guide to interest invokers.](#)



Assignments

- Assignment 1
- Assignment 2



Assignment 1

Create a web page that should display your basic information, with ALT SCHOOL ID, a biography, relevant details explaining why you joined the program, and your goals for the School of Engineering program. Implement a fragment identifier feature to enable scrolling back to the top of the page. Create a folder and name the file index.html

Create a web page consisting of a form that mirrors the ALT SCHOOL application form (<https://portal.altschoolafrica.com/auth/create-account>), name the file form.html, connect the form.html to your first assignment via global navigation ensuring both files are in the folder of the first assignment, which should be at the top of both pages. You should create a footer with your address and a copyright. It must also be accessible, incorporating appropriate semantic HTML tags.

```
1 index.html
2 form.html
```

NB: Strictly without CSS. Submission details will be sent to you in due time.



Assignment 2

Build additional two web pages , the first is going to be a table about all the courses and schools existing at AltSchool Africa, with School of engineering we have frontend engineering, backend engineering, cloud engineering and cybersecurity. School of Product - Product Design, Marketing, Management. School of Data - Data Analysis, Science, Engineering.

School	Courses		
SOE	Frontend	Backend	Cloud
SOD	Design	Marketing	Marketing

Second page is a page where you use all the media element existing in html to create powerful message about yourself. Specially use picture element with more that 3 sources with responsiveness for mobile, tablet and laptop.

```
1 table.html
2 media.html
```



Terminal

- File and Directory Management:

`ls` - List files and directories. `cd` - Change directory. `mkdir` - Create a new directory. `touch` - Create an empty file. `cp` - Copy files or directories. `mv` - Move or rename files or directories. `rm` - Remove files or directories (with caution).

- Viewing and Editing Files:

`cat`, `less` - View file contents. `nano`, `vi`, `vim` - Basic introduction to terminal-based text editors.

- Others

`echo` - Print text to the terminal, `man` - Display the manual for a command, `clear` - Clear the terminal screen, `chmod` - Change file permissions, `chown` - Change file ownership.



Git

- Version Control: A system that records changes to files over time, allowing you to recall specific versions later.
- Distributed Version Control System (DVCS): A version control system that allows multiple developers to work on a project simultaneously.
- Git: A popular DVCS that tracks changes to files and directories, allowing you to collaborate with others and manage your project's history.
-  GitHub: A web-based platform that hosts Git repositories and provides collaboration tools for developers.
- Repository (Repo): A directory that contains your project files and a .git directory, which tracks the history of changes.



- Clone: Creating a copy of an existing Git repository.
- Commit: A snapshot of changes in the repository. It represents a point in the history of your project.
- Branch: A separate line of development, allowing you to work on different features or fixes without affecting the main codebase.
- Merge: Combining changes from one branch into another.
- Remote: A version of your project hosted on the internet or network (e.g., GitHub, GitLab).
- Pull: Fetching changes from a remote repository and merging them into your local repository.
- Push: Sending your commits to a remote repository.
- Pull Request: A request to merge changes from one branch into another.
- Fork: Creating a personal copy of a repository on GitHub.
- Issue: A task, bug, or feature request associated with a repository on GitHub.
- Markdown: A lightweight markup language used to format text on GitHub.
- README: A file that provides information about a project, typically written in Markdown.



Git Commands

- `git init` - Initialize a new Git repository.
- `git clone` - Clone a repository into a new directory.
- `git add` - Add changes to the staging area.
- `git commit` - Record changes to the repository.
- `git status` - Show the working tree status.
- `git log` - Show commit logs.
- `git branch` - List, create, or delete branches.
- `git checkout` - Switch branches or restore working tree files.
- `git merge` - Join two or more development histories together.



- `git pull` - Fetch from and integrate with another repository or a local branch.
- `git push` - Update remote refs along with associated objects.
- `git remote` - Manage set of tracked repositories.
- `git fetch` - Download objects and refs from another repository.
- `git reset` - Reset current HEAD to the specified state.
- `git rebase` - Reapply commits on top of another base tip.
- `git revert` - Revert some existing commits.
- `git stash` - Stash the changes in a dirty working directory away.
- `git tag` - Create, list, delete or verify a tag object signed with GPG.



Git maintains three local storages on our machine and one on a remote server like Github, which means that our code can be found in four places

- Working directory: where we edit files
- Staging area: a temporary location where files are kept for the next commit
- Local repository: contains the code that has been committed
- Remote repository: the remote server that stores the code



GitHub

-  GitHub is a web-based platform that hosts Git repositories and provides collaboration tools for developers. It allows you to store, manage, and share your code with others.
-  GitHub provides features such as pull requests, issues, and project boards to help you collaborate with your team and manage your projects more effectively.
- You can use  GitHub to host your code, track changes, and work on projects with other developers. It's a powerful tool for version control and project management.

Creating a Repository

- Either push an existing local repository to GitHub or create a new repository on  GitHub and clone it to your local machine.

```
1  git clone <repository-url>
2  gh repo create <repository-name>
3  git remote add origin <repository-url>
4  git push
```



Pull Requests

- A pull request is a way to propose changes to a repository on GitHub. It allows you to submit your changes for review and merge them into the main branch.
- When you create a pull request, you can compare the changes between two branches and request feedback from other developers. You can also discuss the changes, make additional commits, and address any feedback before merging the changes.

Creating a Pull Request

- To create a pull request, push your changes to a branch on GitHub and then open a pull request from that branch to the main branch.

```
1 git push origin <branch-name>
2 gh pr create
3 gh pr list
4 gh pr checkout <pull-request-number>
```



Open Source

- Open source software is software that is freely available to use, modify, and distribute. It is developed collaboratively by a community of developers who contribute their time and expertise to improve the software. This software will be used by individuals, businesses, and organizations around the world to build websites, applications, and other software products.
- Open source projects are typically hosted on platforms like GitHub, GitLab, and Bitbucket, where developers can access the source code, report issues, and contribute to the project.

Contributing and Creating Open Source Projects

- You can contribute to open source projects by fixing bugs, adding new features, and improving documentation. You can also create your own open source projects and share them with the community.
- Create tools, libraries, and frameworks that solve common problems and help other developers build better software. Open source projects can be a great way to showcase your skills, collaborate with others, and give back to the community.



Contributors

■  RIDWANADEBOSIN

■  OLUBEBE